Design Patterns in Python

A Complete Textbook — 25 Patterns with Real-World Examples

Contents

Design Patterns in Python — A Complete Textbook **2**
Table of Contents . . . 2

Part I: Behavioral Patterns **4**
Chapter 1: Strategy Pattern . . . 4
Chapter 2: Observer Pattern . . . 8
Chapter 3: Command Pattern . . . 14
Chapter 4: State Pattern . . . 20
Chapter 5: Template Method Pattern . . . 27
Chapter 6: Iterator Pattern . . . 32
Chapter 7: Chain of Responsibility Pattern . . . 37
Chapter 8: Mediator Pattern . . . 42
Chapter 9: Memento Pattern . . . 47
Chapter 10: Visitor Pattern . . . 53

Part II: Creational Patterns **60**
Chapter 11: Factory Method Pattern . . . 60
Chapter 12: Abstract Factory Pattern . . . 65
Chapter 13: Builder Pattern . . . 70
Chapter 14: Singleton Pattern . . . 76
Chapter 15: Prototype Pattern . . . 81

Part III: Structural Patterns **87**
Chapter 16: Decorator Pattern . . . 87
Chapter 17: Adapter Pattern . . . 92
Chapter 18: Facade Pattern . . . 97
Chapter 19: Composite Pattern . . . 102
Chapter 20: Proxy Pattern . . . 107
Chapter 21: Bridge Pattern . . . 113
Chapter 22: Flyweight Pattern . . . 118

Part IV: Architectural Patterns (Bonus) **124**
Chapter 23: Repository Pattern . . . 124
Chapter 24: Dependency Injection . . . 130
Chapter 25: Event Sourcing . . . 136

Appendix: Pattern Quick Reference **143**

Design Patterns in Python — A Complete Textbook

25 Patterns with Real-World Examples

Table of Contents

Part I: Behavioral Patterns

1. Strategy Pattern
2. Observer Pattern
3. Command Pattern
4. State Pattern
5. Template Method Pattern
6. Iterator Pattern
7. Chain of Responsibility Pattern
8. Mediator Pattern
9. Memento Pattern
10. Visitor Pattern

Part II: Creational Patterns

11. Factory Method Pattern
12. Abstract Factory Pattern
13. Builder Pattern
14. Singleton Pattern
15. Prototype Pattern

Part III: Structural Patterns

16. Decorator Pattern
17. Adapter Pattern
18. Facade Pattern
19. Composite Pattern
20. Proxy Pattern
21. Bridge Pattern
22. Flyweight Pattern

Part IV: Architectural Patterns (Bonus)

23. Repository Pattern
24. Dependency Injection
25. Event Sourcing

Appendix: Pattern Quick Reference

Part I: Behavioral Patterns

Chapter 1: Strategy Pattern

Category: Behavioral

Intent

Define a family of algorithms, encapsulate each one, and make them interchangeable at runtime.

Real-World Analogy

Think of paying at a store. You can pay by cash, credit card, or mobile payment. The store doesn't care HOW you pay — it just asks you to "pay". Each payment method is a "strategy".

The Problem

Imagine you're building a navigation app. Users can travel by car, bicycle, or walking. Each mode calculates the route differently. How do you handle this without a mess of `if/elif` statements?

```
"""
============================================================
 BAD EXAMPLE: Without the Strategy Pattern
============================================================

This is how beginners often write it. It works, but it becomes
a nightmare as the project grows.

Problems:
    1. Every new transport mode -> modify this class (violates Open/Closed)
    2. The class keeps growing with if/elif chains
    3. Hard to test individual routing algorithms
    4. If car routing has a bug, you risk breaking bicycle routing too
============================================================
```

```python
"""


class BadNavigator:
    def navigate(self, mode: str, origin: str, destination: str) -> str:
        if mode == "car":
            return f"Driving from {origin} to {destination}: Use highways."
        elif mode == "bicycle":
            return f"Cycling from {origin} to {destination}: Use bike lanes."
        elif mode == "walking":
            return f"Walking from {origin} to {destination}: Use sidewalks."
        # Imagine adding 10 more modes here...
        # elif mode == "bus":
        # elif mode == "train":
        # elif mode == "scooter":
        # elif mode == "helicopter":
        # This method becomes HUGE and fragile!
        else:
            raise ValueError(f"Unknown mode: {mode}")


if __name__ == "__main__":
    nav = BadNavigator()
    print(nav.navigate("car", "Home", "Office"))
    print(nav.navigate("bicycle", "Home", "Office"))
    print(nav.navigate("walking", "Home", "Office"))

    print()
    print("This works... but what happens when you need to:")
    print("  - Add 10 more transport modes?")
    print("  - Test car routing independently?")
    print("  - Let users create custom routing algorithms?")
    print("  - Reuse bicycle routing in another app?")
    print()
    print("-> The Strategy pattern solves ALL of these problems.")
```

The Solution

Encapsulate each algorithm in its own class, and let the context delegate to whichever strategy it holds.

```python
"""
================================================================
 DESIGN PATTERN #1: STRATEGY
================================================================

Category: Behavioral
```

```python
Intent:   Define a family of algorithms, encapsulate each one,
          and make them interchangeable at runtime.

Real-world analogy:
    Think of paying at a store. You can pay by cash, credit card,
    or mobile payment. The store doesn't care HOW you pay —
    it just asks you to "pay". Each payment method is a "strategy".

When to use:
    - You have multiple ways to do the same thing
    - You want to switch algorithms at runtime
    - You want to avoid long if/elif/else chains

==============================================================
"""

from abc import ABC, abstractmethod


# ------------------------------------------------------------
# STEP 1: Define the Strategy interface
# ------------------------------------------------------------
# This is the "contract" — every strategy must implement this.

class RouteStrategy(ABC):
    @abstractmethod
    def calculate_route(self, origin: str, destination: str) -> str:
        pass


# ------------------------------------------------------------
# STEP 2: Implement concrete strategies
# ------------------------------------------------------------
# Each class = one algorithm. Clean, isolated, testable.

class CarRoute(RouteStrategy):
    def calculate_route(self, origin: str, destination: str) -> str:
        return f"[CAR] Driving route from {origin} to {destination}: Use highways, avoi


class BicycleRoute(RouteStrategy):
    def calculate_route(self, origin: str, destination: str) -> str:
        return f"[BIKE] Cycling route from {origin} to {destination}: Use bike lanes, a


class WalkingRoute(RouteStrategy):
    def calculate_route(self, origin: str, destination: str) -> str:
```

```
        return f"[WALK] Walking route from {origin} to {destination}: Use sidewalks, ta


# ------------------------------------------------------------
# STEP 3: The Context class — uses a strategy
# ------------------------------------------------------------
# The Navigator doesn't know HOW the route is calculated.
# It just delegates to whatever strategy it's given.

class Navigator:
    def __init__(self, strategy: RouteStrategy):
        self._strategy = strategy

    def set_strategy(self, strategy: RouteStrategy):
        """Change the strategy at runtime!"""
        self._strategy = strategy

    def navigate(self, origin: str, destination: str) -> str:
        return self._strategy.calculate_route(origin, destination)


# ------------------------------------------------------------
# STEP 4: See it in action
# ------------------------------------------------------------

if __name__ == "__main__":
    # Create a navigator with a car strategy
    nav = Navigator(CarRoute())
    print(nav.navigate("Home", "Office"))

    # User switches to bicycle — just swap the strategy!
    nav.set_strategy(BicycleRoute())
    print(nav.navigate("Home", "Office"))

    # User switches to walking
    nav.set_strategy(WalkingRoute())
    print(nav.navigate("Home", "Office"))

    print()
    print("=" * 60)
    print("KEY TAKEAWAYS:")
    print("=" * 60)
    print("1. Navigator doesn't contain any routing logic itself.")
    print("2. Adding a new transport mode = just add a new class.")
    print("3. No if/elif/else chains. No modification of existing code.")
    print("4. This follows the Open/Closed Principle:")
    print("   -> Open for extension, Closed for modification.")
```

Structure

```
+-------------+       uses        +-----------------+
|  Navigator   | ----------------> |  RouteStrategy    | (interface)
|  (Context)   |                   |  calculate_route()|
+-------------+                  +-----------------+
                                     ^    ^    ^
                                     |    |    |
                              +-----+    |    +-----+
                              |          |          |
                        +---------+ +--------+ +--------+
                        |CarRoute | |Bicycle | |Walking |
                        |         | |Route   | |Route   |
                        +---------+ +--------+ +--------+
```

The 3 Core Pieces

1. **Strategy** (interface) — defines what all algorithms must do
2. **Concrete Strategies** — each implements one algorithm
3. **Context** — holds a reference to a strategy and delegates work to it

When to Use

- You have multiple ways to do the same thing
- You want to switch algorithms at runtime
- You want to avoid long if/elif/else chains

Key Takeaways

1. The Context class doesn't contain any algorithm logic itself
2. Adding a new algorithm = just add a new class
3. No if/elif/else chains. No modification of existing code
4. Follows the **Open/Closed Principle**: open for extension, closed for modification

---

Chapter 2: Observer Pattern

Category: Behavioral

Intent

Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.

Real-World Analogy

YouTube subscriptions. You subscribe to a channel, and you get notified when new content is uploaded. You don't have to keep checking. You can also unsubscribe anytime.

The Problem

Imagine you run an online store. When a product comes back in stock, you need to notify customers who are waiting for it. You could check every customer manually… or you could let them **subscribe** and get notified automatically.

```python
"""
==============================================================
 BAD EXAMPLE: Without the Observer Pattern
==============================================================

An online store that notifies customers when a product is back in stock.

Problems:
    1. The Store class must KNOW about every notification method
    2. Adding SMS notification -> modify the Store class
    3. Adding push notification -> modify the Store class again
    4. Store is tightly coupled to all notification logic
    5. Can't add/remove subscribers at runtime
==============================================================
"""


class BadStore:
    def __init__(self):
        self.customer_emails = []
        self.customer_phones = []

    def add_email_customer(self, email: str):
        self.customer_emails.append(email)

    def add_phone_customer(self, phone: str):
        self.customer_phones.append(phone)

    def restock(self, product: str):
        print(f"\n[PKG] '{product}' is back in stock!")

        # Store has to handle EVERY notification type itself
        for email in self.customer_emails:
            print(f"  [EMAIL] Sending email to {email}: '{product}' is available!")

        for phone in self.customer_phones:
            print(f"  [SMS] Sending SMS to {phone}: '{product}' is available!")
```

```
        # Want to add push notifications? Slack alerts? Webhook calls?
        # You have to modify THIS class every single time!


if __name__ == "__main__":
    store = BadStore()
    store.add_email_customer("alice@mail.com")
    store.add_email_customer("bob@mail.com")
    store.add_phone_customer("010-1234-5678")

    store.restock("PlayStation 6")

    print()
    print("Problems with this approach:")
    print("  - Store knows about emails, phones, and every future channel")
    print("  - Adding a new channel = modifying the Store class")
    print("  - Can't reuse notification logic elsewhere")
    print("  -> The Observer pattern fixes this.")
```

The Solution

Let observers subscribe to the subject. When the subject's state changes, it notifies all observers automatically.

```
"""
================================================================
 DESIGN PATTERN #2: OBSERVER
================================================================

Category: Behavioral
Intent:   Define a one-to-many dependency between objects so that
          when one object changes state, all its dependents are
          notified and updated automatically.

Real-world analogy:
    YouTube subscriptions. You subscribe to a channel, and you
    get notified when new content is uploaded. You can also
    unsubscribe anytime.

Key terms:
    - Subject (Publisher):  the object being watched
    - Observer (Subscriber): the object that reacts to changes

When to use:
    - When changes in one object should trigger updates in others
    - When you don't know in advance how many objects need updating
    - When you want loose coupling between the sender and receivers
```

```
==============================================================
"""

from abc import ABC, abstractmethod


# ---------------------------------------------------------
# STEP 1: Define the Observer interface
# ---------------------------------------------------------
# Any class that wants to receive notifications must implement this.

class Observer(ABC):
    @abstractmethod
    def update(self, product: str) -> None:
        pass


# ---------------------------------------------------------
# STEP 2: Define the Subject (Publisher)
# ---------------------------------------------------------
# The Subject doesn't know WHO is observing — it just knows
# that observers exist and they have an update() method.

class Store:
    def __init__(self):
        self._observers: list[Observer] = []

    def subscribe(self, observer: Observer) -> None:
        self._observers.append(observer)
        print(f"  [v] {observer} subscribed.")

    def unsubscribe(self, observer: Observer) -> None:
        self._observers.remove(observer)
        print(f"  [x] {observer} unsubscribed.")

    def notify_all(self, product: str) -> None:
        for observer in self._observers:
            observer.update(product)

    def restock(self, product: str) -> None:
        print(f"\n[PKG] '{product}' is back in stock!")
        self.notify_all(product)


# ---------------------------------------------------------
# STEP 3: Implement concrete observers
# ---------------------------------------------------------
```

```python
# Each observer decides HOW to react. The Store doesn't care.

class EmailSubscriber(Observer):
    def __init__(self, email: str):
        self.email = email

    def update(self, product: str) -> None:
        print(f"  [EMAIL] Email to {self.email}: '{product}' is now available!")

    def __str__(self):
        return f"EmailSubscriber({self.email})"


class SMSSubscriber(Observer):
    def __init__(self, phone: str):
        self.phone = phone

    def update(self, product: str) -> None:
        print(f"  [SMS] SMS to {self.phone}: '{product}' is now available!")

    def __str__(self):
        return f"SMSSubscriber({self.phone})"


class SlackSubscriber(Observer):
    """
    Look how easy it is to add a new notification channel!
    No modification to the Store class at all.
    """
    def __init__(self, channel: str):
        self.channel = channel

    def update(self, product: str) -> None:
        print(f"  [CHAT] Slack #{self.channel}: '{product}' is now available!")

    def __str__(self):
        return f"SlackSubscriber(#{self.channel})"


# ------------------------------------------------------------
# STEP 4: See it in action
# ------------------------------------------------------------

if __name__ == "__main__":
    store = Store()

    # Customers subscribe
```

```python
    print("=== Subscribing ===")
    alice = EmailSubscriber("alice@mail.com")
    bob = SMSSubscriber("010-1234-5678")
    dev_team = SlackSubscriber("dev-alerts")

    store.subscribe(alice)
    store.subscribe(bob)
    store.subscribe(dev_team)

    # Product restocked -> everyone gets notified
    store.restock("PlayStation 6")

    # Bob unsubscribes — he already bought one
    print("\n=== Bob unsubscribes ===")
    store.unsubscribe(bob)

    # Another restock -> only remaining subscribers are notified
    store.restock("Nintendo Switch 2")

    print()
    print("=" * 60)
    print("KEY TAKEAWAYS:")
    print("=" * 60)
    print("1. Store doesn't know about email, SMS, or Slack specifics.")
    print("2. Adding a new channel = just create a new Observer class.")
    print("3. Subscribers can join/leave at runtime.")
    print("4. Store and Observers are loosely coupled.")
    print("5. This is the foundation of event-driven programming!")
```

Structure

```
                        subscribes
  EmailSubscriber  --------------+
  SMSSubscriber    --------------+
  SlackSubscriber  --------------+
                                 v
                          +------------+
                          |   Store    |
                          | (Subject)  |
                          |            |
                          | restock()  |---> notify_all()
                          |            |       |
                          +------------+       |
                                               v
                                    calls update() on
                                    every subscriber
```

When to Use

- When changes in one object should trigger updates in others
- When you don't know in advance how many objects need updating
- When you want loose coupling between the sender and receivers

Key Takeaways

1. The Subject doesn't know about email, SMS, or Slack specifics
2. Adding a new notification channel = just create a new Observer class
3. Subscribers can join and leave at runtime
4. Subject and Observers are loosely coupled
5. This is the foundation of event-driven programming

Strategy vs Observer — Comparison

	Strategy	Observer
Relationship	1-to-1 (context to strategy)	1-to-many (subject to observers)
Purpose	Swap **one** algorithm	Notify **many** listeners
Direction	Context calls strategy	Subject broadcasts to observers

---

Chapter 3: Command Pattern

Category: Behavioral

Intent

Encapsulate a request as an object, thereby allowing you to parameterize, queue, log, and undo/redo operations.

Real-World Analogy

A restaurant order. You don't cook the food yourself — you write an order (command object), hand it to the waiter (invoker), who passes it to the chef (receiver). The order can be queued, cancelled, or logged.

The Problem

You're building a text editor. Users can type, undo, redo, and record macros. If actions are just direct method calls, there's no history, no way to reverse them, and no way to replay them.

```
"""
============================================================
```

```python
 BAD EXAMPLE: Without the Command Pattern
================================================================

A simple text editor where actions are directly executed.

Problems:
    1. No way to undo/redo — actions are fire-and-forget
    2. No history of what was done
    3. Can't queue, schedule, or replay actions
    4. Adding undo means polluting the Editor with undo logic
       for every possible action
================================================================
"""


class BadTextEditor:
    def __init__(self):
        self.text = ""

    def type_text(self, text: str):
        self.text += text
        print(f"  Typed: '{text}' -> \"{self.text}\"")

    def delete_last(self, count: int):
        deleted = self.text[-count:]
        self.text = self.text[:-count]
        print(f"  Deleted: '{deleted}' -> \"{self.text}\"")

    # Want undo? You'd have to track state for EACH method.
    # What about redo? What about macros?
    # This class would explode in complexity.


if __name__ == "__main__":
    editor = BadTextEditor()
    editor.type_text("Hello")
    editor.type_text(" World")
    editor.delete_last(5)

    print(f"\nFinal text: \"{editor.text}\"")
    print()
    print("Now try to undo the delete... you can't!")
    print("There's no history, no way to reverse actions.")
    print("-> The Command pattern fixes this.")
```

The Solution

Turn each action into an object that knows how to execute AND undo itself.

```
"""
==============================================================
 DESIGN PATTERN #3: COMMAND
==============================================================

Category: Behavioral
Intent:   Encapsulate a request as an object, thereby allowing you
          to parameterize, queue, log, and undo/redo operations.

Real-world analogy:
    A restaurant order. You don't cook the food yourself — you
    write an order (command object), hand it to the waiter (invoker),
    who passes it to the chef (receiver). The order can be queued,
    cancelled, or logged.

Key terms:
    - Command:   the action object (what to do + how to undo)
    - Receiver:  the object that actually performs the work
    - Invoker:   the object that triggers commands (and keeps history)

When to use:
    - You need undo/redo functionality
    - You want to queue or schedule operations
    - You want to log a history of actions
    - You want to support macros (replay a sequence of commands)

==============================================================
"""

from abc import ABC, abstractmethod


# ----------------------------------------------------------
# STEP 1: The Receiver — does the actual work
# ----------------------------------------------------------
# This is just a plain object. It doesn't know about commands.

class TextEditor:
    def __init__(self):
        self.text = ""

    def insert(self, text: str, position: int) -> None:
        self.text = self.text[:position] + text + self.text[position:]

    def delete(self, position: int, count: int) -> str:
```

```python
        deleted = self.text[position:position + count]
        self.text = self.text[:position] + self.text[position + count:]
        return deleted


# ---------------------------------------------------------
# STEP 2: The Command interface
# ---------------------------------------------------------
# Every command must know how to execute AND how to undo.

class Command(ABC):
    @abstractmethod
    def execute(self) -> None:
        pass

    @abstractmethod
    def undo(self) -> None:
        pass


# ---------------------------------------------------------
# STEP 3: Concrete Commands
# ---------------------------------------------------------
# Each command stores everything it needs to execute AND undo.

class TypeCommand(Command):
    def __init__(self, editor: TextEditor, text: str):
        self.editor = editor
        self.text = text
        self.position = len(editor.text)  # insert at end

    def execute(self) -> None:
        self.editor.insert(self.text, self.position)
        print(f"  [WRITE]  Typed '{self.text}' -> \"{self.editor.text}\"")

    def undo(self) -> None:
        self.editor.delete(self.position, len(self.text))
        print(f"  [UNDO]  Undo type '{self.text}' -> \"{self.editor.text}\"")


class DeleteCommand(Command):
    def __init__(self, editor: TextEditor, count: int):
        self.editor = editor
        self.count = count
        self.position = len(editor.text) - count
        self.deleted_text = ""  # saved during execute for undo
```

```python
    def execute(self) -> None:
        self.deleted_text = self.editor.delete(self.position, self.count)
        print(f"  [DEL]  Deleted '{self.deleted_text}' -> \"{self.editor.text}\"")

    def undo(self) -> None:
        self.editor.insert(self.deleted_text, self.position)
        print(f"  [UNDO]  Undo delete '{self.deleted_text}' -> \"{self.editor.text}\"")


# ------------------------------------------------------------
# STEP 4: The Invoker — manages command history
# ------------------------------------------------------------
# This is the "brain" that executes, undoes, and redoes.

class CommandManager:
    def __init__(self):
        self.history: list[Command] = []
        self.redo_stack: list[Command] = []

    def execute(self, command: Command) -> None:
        command.execute()
        self.history.append(command)
        self.redo_stack.clear()  # new action clears redo history

    def undo(self) -> None:
        if not self.history:
            print("  [\!]  Nothing to undo!")
            return
        command = self.history.pop()
        command.undo()
        self.redo_stack.append(command)

    def redo(self) -> None:
        if not self.redo_stack:
            print("  [\!]  Nothing to redo!")
            return
        command = self.redo_stack.pop()
        command.execute()
        self.history.append(command)


# ------------------------------------------------------------
# STEP 5: See it in action
# ------------------------------------------------------------

if __name__ == "__main__":
    editor = TextEditor()
```

```python
    manager = CommandManager()

    print("=== Typing ===")
    manager.execute(TypeCommand(editor, "Hello"))
    manager.execute(TypeCommand(editor, " Beautiful"))
    manager.execute(TypeCommand(editor, " World"))

    print("\n=== Undo (3 times) ===")
    manager.undo()  # undo " World"
    manager.undo()  # undo " Beautiful"
    manager.undo()  # undo "Hello"

    print("\n=== Redo (2 times) ===")
    manager.redo()  # redo "Hello"
    manager.redo()  # redo " Beautiful"

    print("\n=== Type more, then delete ===")
    manager.execute(TypeCommand(editor, " Day"))
    manager.execute(DeleteCommand(editor, 4))  # delete " Day"

    print("\n=== Undo delete ===")
    manager.undo()  # undo the delete -> " Day" comes back

    print(f"\n[FILE] Final text: \"{editor.text}\"")

    print()
    print("=" * 60)
    print("KEY TAKEAWAYS:")
    print("=" * 60)
    print("1. Each action is an object that knows how to do AND undo.")
    print("2. The CommandManager keeps a history stack for undo/redo.")
    print("3. New actions (e.g., BoldCommand) = just add a new class.")
    print("4. The Editor (receiver) stays simple — no undo logic.")
    print("5. Commands can be queued, logged, serialized, or replayed.")
```

Structure

```
+----------------+  executes   +---------------+  acts on   +------------+
| CommandManager | -----------> |   Command     | ----------> | TextEditor |
|   (Invoker)    |             |  execute()    |            | (Receiver) |
|                |             |  undo()       |            |            |
| history: []    |             +---------------+            +------------+
| redo_stack: [] |                ^         ^
+----------------+                |         |
                          +------+         +------+
                          |                       |
                     +-----------+          +------------+
```

```
|TypeCommand|          |DeleteCommand|
+-----------+          +------------+
```

When to Use

- You need undo/redo functionality
- You want to queue or schedule operations
- You want to log a history of actions
- You want to support macros (replay a sequence of commands)

Key Takeaways

1. Each action is an object that knows how to do AND undo
2. The CommandManager keeps a history stack for undo/redo
3. New actions (e.g., BoldCommand) = just add a new class
4. The Receiver stays simple — no undo logic in it
5. Commands can be queued, logged, serialized, or replayed

Comparison So Far

Pattern	Relationship	Core Idea
Strategy	1-to-1	Swap one algorithm
Observer	1-to-many	Notify many listeners
Command	1-to-1 (with history)	Turn actions into objects

Chapter 4: State Pattern

Category: Behavioral

Intent

Allow an object to alter its behavior when its internal state changes. The object will appear to change its class.

Real-World Analogy

A traffic light. When it's green, cars go. When it's red, cars stop. The light doesn't use if/elif — it simply switches between state objects.

The Problem

A vending machine has multiple states: idle, has_money, dispensing. Every method needs if/elif checks for the current state. Adding a new state means modifying every

method.

```
"""
================================================================
 BAD EXAMPLE: Without the State Pattern
================================================================

A vending machine with multiple states: idle, has_money, dispensing.

Problems:
    1. Every method has if/elif checks for the current state
    2. Adding a new state -> modify EVERY method
    3. Easy to forget a state transition -> bugs
    4. The class grows huge as states multiply
================================================================
"""


class BadVendingMachine:
    def __init__(self):
        self.state = "idle"
        self.balance = 0

    def insert_money(self, amount: int):
        if self.state == "idle":
            self.balance = amount
            self.state = "has_money"
            print(f"  Inserted ${amount}. Balance: ${self.balance}")
        elif self.state == "has_money":
            self.balance += amount
            print(f"  Added ${amount}. Balance: ${self.balance}")
        elif self.state == "dispensing":
            print("  [\!] Please wait, dispensing in progress...")
        # elif self.state == "out_of_stock": ...
        # elif self.state == "maintenance": ...
        # Every new state = another elif HERE and in EVERY method!

    def select_product(self, price: int):
        if self.state == "idle":
            print("  [\!] Please insert money first.")
        elif self.state == "has_money":
            if self.balance >= price:
                self.state = "dispensing"
                print(f"  Dispensing product (${price})...")
                self.balance -= price
                self.state = "idle"
                if self.balance > 0:
                    print(f"  Change: ${self.balance}")
```

```
                    self.balance = 0
            else:
                print(f"  [\!] Not enough money. Need ${price}, have ${self.balance}")
        elif self.state == "dispensing":
            print("  [\!] Please wait, dispensing in progress...")

    def cancel(self):
        if self.state == "idle":
            print("  Nothing to cancel.")
        elif self.state == "has_money":
            print(f"  Returning ${self.balance}.")
            self.balance = 0
            self.state = "idle"
        elif self.state == "dispensing":
            print("  [\!] Cannot cancel during dispensing.")


if __name__ == "__main__":
    machine = BadVendingMachine()
    machine.select_product(5)
    machine.insert_money(3)
    machine.insert_money(2)
    machine.select_product(4)

    print()
    print("This works, but every method is full of if/elif state checks.")
    print("Adding a 'maintenance' or 'out_of_stock' state would mean")
    print("editing EVERY method. -> The State pattern fixes this.")
```

The Solution

Encapsulate each state in its own class. The state object handles behavior and decides when to transition.

```
"""
================================================================
 DESIGN PATTERN #4: STATE
================================================================

Category: Behavioral
Intent:   Allow an object to alter its behavior when its internal
          state changes. The object will appear to change its class.

Real-world analogy:
    A traffic light. When it's green, cars go. When it's red,
    cars stop. The light itself doesn't have if/elif logic —
    it simply switches between state objects.
```

```
When to use:
    - An object behaves differently depending on its state
    - You have many if/elif checks on a state variable
    - State transitions are complex and error-prone

==============================================================
"""

from abc import ABC, abstractmethod


# ---------------------------------------------------------
# STEP 1: Define the State interface
# ---------------------------------------------------------
# Each state handles the SAME set of actions differently.

class VendingState(ABC):
    @abstractmethod
    def insert_money(self, machine: "VendingMachine", amount: int) -> None:
        pass

    @abstractmethod
    def select_product(self, machine: "VendingMachine", price: int) -> None:
        pass

    @abstractmethod
    def cancel(self, machine: "VendingMachine") -> None:
        pass


# ---------------------------------------------------------
# STEP 2: Implement concrete states
# ---------------------------------------------------------
# Each state class is clean and focused on ONE state's behavior.

class IdleState(VendingState):
    def insert_money(self, machine: "VendingMachine", amount: int) -> None:
        machine.balance = amount
        print(f"  [526925] Inserted ${amount}. Balance: ${machine.balance}")
        machine.set_state(HasMoneyState())

    def select_product(self, machine: "VendingMachine", price: int) -> None:
        print("  [\!] Please insert money first.")

    def cancel(self, machine: "VendingMachine") -> None:
        print("  Nothing to cancel.")
```

```python
    def __str__(self):
        return "IdleState"


class HasMoneyState(VendingState):
    def insert_money(self, machine: "VendingMachine", amount: int) -> None:
        machine.balance += amount
        print(f"  [526925] Added ${amount}. Balance: ${machine.balance}")

    def select_product(self, machine: "VendingMachine", price: int) -> None:
        if machine.balance >= price:
            machine.set_state(DispensingState())
            machine.state.dispense(machine, price)
        else:
            print(f"  [\!] Not enough. Need ${price}, have ${machine.balance}")

    def cancel(self, machine: "VendingMachine") -> None:
        print(f"  [526925] Returning ${machine.balance}.")
        machine.balance = 0
        machine.set_state(IdleState())

    def __str__(self):
        return "HasMoneyState"


class DispensingState(VendingState):
    def insert_money(self, machine: "VendingMachine", amount: int) -> None:
        print("  [\!] Please wait, dispensing in progress...")

    def select_product(self, machine: "VendingMachine", price: int) -> None:
        print("  [\!] Please wait, dispensing in progress...")

    def cancel(self, machine: "VendingMachine") -> None:
        print("  [\!] Cannot cancel during dispensing.")

    def dispense(self, machine: "VendingMachine", price: int) -> None:
        print(f"  [PKG] Dispensing product (${price})...")
        machine.balance -= price
        if machine.balance > 0:
            print(f"  [526925] Change: ${machine.balance}")
            machine.balance = 0
        machine.set_state(IdleState())

    def __str__(self):
        return "DispensingState"
```

```python
# ------------------------------------------------------------
# STEP 3: The Context — delegates everything to current state
# ------------------------------------------------------------

class VendingMachine:
    def __init__(self):
        self.balance = 0
        self.state: VendingState = IdleState()

    def set_state(self, state: VendingState) -> None:
        print(f"    [State: {self.state} -> {state}]")
        self.state = state

    def insert_money(self, amount: int) -> None:
        self.state.insert_money(self, amount)

    def select_product(self, price: int) -> None:
        self.state.select_product(self, price)

    def cancel(self) -> None:
        self.state.cancel(self)


# ------------------------------------------------------------
# STEP 4: See it in action
# ------------------------------------------------------------

if __name__ == "__main__":
    machine = VendingMachine()

    print("=== Try to buy without money ===")
    machine.select_product(5)

    print("\n=== Insert money and buy ===")
    machine.insert_money(3)
    machine.insert_money(4)
    machine.select_product(5)

    print("\n=== Insert money and cancel ===")
    machine.insert_money(10)
    machine.cancel()

    print()
    print("=" * 60)
    print("KEY TAKEAWAYS:")
    print("=" * 60)
    print("1. No if/elif chains — each state is its own class.")
```

```
    print("2. Adding 'MaintenanceState' = just add a new class.")
    print("3. State transitions are explicit and easy to trace.")
    print("4. Each state class is small, focused, and testable.")
    print("5. Very similar to Strategy, but the STATE decides")
    print("   when to transition (not the client).")
```

Structure

```
+----------------+  delegates to  +------------------+
| VendingMachine | --------------> |  VendingState     | (interface)
|   (Context)    |                |  insert_money()   |
|                |                |  select_product() |
| state ---------+                |  cancel()         |
+----------------+                +------------------+
                                     ^    ^    ^
                                     |    |    |
                              +------+    |    +-------+
                              |           |            |
                        +----------+ +----------+ +-----------+
                        |  Idle    | | HasMoney | |Dispensing |
                        |  State   | |  State   | |  State    |
                        +----------+ +----------+ +-----------+
```

State vs Strategy

	Strategy	State
Who switches?	The client sets the strategy	The state switches itself
Awareness	Strategies don't know about each other	States know about transitions
Purpose	Choose one algorithm	Model a state machine

When to Use

- An object behaves differently depending on its state
- You have many if/elif checks on a state variable
- State transitions are complex and error-prone

Key Takeaways

1. No if/elif chains — each state is its own class
2. Adding a new state = just add a new class
3. State transitions are explicit and easy to trace
4. Each state class is small, focused, and testable
5. Very similar to Strategy, but the state decides when to transition

---

Chapter 5: Template Method Pattern

Category: Behavioral

Intent

Define the skeleton of an algorithm in a base class, letting subclasses override specific steps without changing the algorithm's structure.

Real-World Analogy

Building a house. The blueprint defines the order: foundation, walls, roof, interior. Every house follows this order, but a wooden house and a brick house differ in HOW they build the walls.

The Problem

You have data processors for CSV, JSON, XML. They all follow the same workflow: read, process, analyze, report. But each duplicates the workflow structure and shared logic.

```
"""
==============================================================
 BAD EXAMPLE: Without the Template Method Pattern
==============================================================

Data processing pipelines for CSV and JSON files.
Both follow the same steps: read -> process -> analyze -> report.

Problems:
    1. Duplicated structure — both classes repeat the same workflow
    2. If you change the workflow order, you must edit EVERY class
    3. Easy to forget a step or get the order wrong
    4. Common steps (like reporting) are duplicated
==============================================================
"""


class BadCSVProcessor:
    def run(self):
        # Step 1: Read
        print("  Reading CSV file...")
        data = [{"name": "Alice", "score": 90}, {"name": "Bob", "score": 75}]

        # Step 2: Process
        print("  Parsing CSV rows...")
```

```
        processed = [row["score"] for row in data]

        # Step 3: Analyze
        avg = sum(processed) / len(processed)
        print(f"  Average score: {avg}")

        # Step 4: Report
        print(f"  Report: Processed {len(data)} records. Average: {avg}")


class BadJSONProcessor:
    def run(self):
        # Step 1: Read (same structure, different detail)
        print("  Reading JSON file...")
        data = [{"name": "Charlie", "score": 85}, {"name": "Diana", "score": 95}]

        # Step 2: Process
        print("  Parsing JSON objects...")
        processed = [item["score"] for item in data]

        # Step 3: Analyze
        avg = sum(processed) / len(processed)
        print(f"  Average score: {avg}")

        # Step 4: Report (exact duplicate!)
        print(f"  Report: Processed {len(data)} records. Average: {avg}")


if __name__ == "__main__":
    print("=== CSV Processing ===")
    BadCSVProcessor().run()
    print("\n=== JSON Processing ===")
    BadJSONProcessor().run()

    print()
    print("The workflow is identical. Only the 'read' and 'process'")
    print("steps differ. The rest is duplicated code.")
    print("-> The Template Method pattern fixes this.")
```

The Solution

Define the algorithm skeleton once in the base class. Subclasses only override the steps that vary.

```
"""
================================================================
 DESIGN PATTERN #5: TEMPLATE METHOD
```

```
==============================================================

Category: Behavioral
Intent:   Define the skeleton of an algorithm in a base class,
          letting subclasses override specific steps without
          changing the algorithm's structure.

Real-world analogy:
    Building a house. The blueprint (template) defines the order:
    foundation -> walls -> roof -> interior. Every house follows
    this order, but a wooden house and a brick house differ in
    HOW they build walls.

When to use:
    - Multiple classes follow the same algorithm structure
    - Only some steps vary between classes
    - You want to enforce an order of operations

==============================================================
"""

from abc import ABC, abstractmethod


# ---------------------------------------------------------
# STEP 1: The abstract base class with the "template method"
# ---------------------------------------------------------
# run() is the template method — it defines the skeleton.
# Subclasses CANNOT change the order, only the individual steps.

class DataProcessor(ABC):

    def run(self):
        """The template method — defines the algorithm skeleton."""
        data = self.read_data()
        processed = self.process_data(data)
        result = self.analyze(processed)
        self.report(data, result)

    # Abstract steps — subclasses MUST implement these
    @abstractmethod
    def read_data(self) -> list[dict]:
        pass

    @abstractmethod
    def process_data(self, data: list[dict]) -> list:
        pass
```

```python
    # Concrete steps — shared by all subclasses (no duplication!)
    def analyze(self, values: list) -> float:
        avg = sum(values) / len(values)
        print(f"  [STAT] Average: {avg}")
        return avg

    def report(self, data: list, result: float) -> None:
        print(f"  [RPT] Report: Processed {len(data)} records. Average: {result}")


# ------------------------------------------------------------
# STEP 2: Concrete implementations — only override what differs
# ------------------------------------------------------------

class CSVProcessor(DataProcessor):
    def read_data(self) -> list[dict]:
        print("  [DIR] Reading CSV file...")
        return [{"name": "Alice", "score": 90}, {"name": "Bob", "score": 75}]

    def process_data(self, data: list[dict]) -> list:
        print("  [PROC] Parsing CSV rows...")
        return [row["score"] for row in data]


class JSONProcessor(DataProcessor):
    def read_data(self) -> list[dict]:
        print("  [DIR] Reading JSON file...")
        return [{"name": "Charlie", "score": 85}, {"name": "Diana", "score": 95}]

    def process_data(self, data: list[dict]) -> list:
        print("  [PROC] Parsing JSON objects...")
        return [item["score"] for item in data]


class XMLProcessor(DataProcessor):
    """Adding a new format is trivial — just implement the varying steps."""
    def read_data(self) -> list[dict]:
        print("  [DIR] Reading XML file...")
        return [{"name": "Eve", "score": 88}, {"name": "Frank", "score": 92}]

    def process_data(self, data: list[dict]) -> list:
        print("  [PROC] Parsing XML elements...")
        return [item["score"] for item in data]


# ------------------------------------------------------------
# STEP 3: See it in action
```

```python
# ------------------------------------------------------------

if __name__ == "__main__":
    for name, processor in [("CSV", CSVProcessor()),
                            ("JSON", JSONProcessor()),
                            ("XML", XMLProcessor())]:
        print(f"=== {name} Processing ===")
        processor.run()
        print()

    print("=" * 60)
    print("KEY TAKEAWAYS:")
    print("=" * 60)
    print("1. The algorithm structure (read->process->analyze->report)")
    print("   is defined ONCE in the base class.")
    print("2. Subclasses only override the steps that vary.")
    print("3. Common logic (analyze, report) is NOT duplicated.")
    print("4. The order of steps is enforced — can't accidentally skip one.")
    print("5. Hollywood Principle: 'Don't call us, we'll call you.'")
    print("   The base class calls the subclass, not the other way around.")
```

Structure

```
+-----------------------------+
|     DataProcessor (ABC)     |
|                             |
|  run()  <- template method   |
|    1. read_data()   abstract |
|    2. process_data() abstract|
|    3. analyze()     concrete |
|    4. report()      concrete |
+-----------------------------+
          ^         ^
          |         |
   +------------+ +-------------+
   |CSVProcessor| |JSONProcessor|
   | read_data()| | read_data() |
   | process()  | | process()   |
   +------------+ +-------------+
```

When to Use

- Multiple classes follow the same algorithm structure
- Only some steps vary between classes
- You want to enforce an order of operations

Key Takeaways

1. The algorithm skeleton is defined ONCE in the base class
2. Subclasses only override the steps that vary
3. Common logic is NOT duplicated
4. The order of steps is enforced
5. Hollywood Principle: "Don't call us, we'll call you"

Chapter 6: Iterator Pattern

Category: Behavioral

Intent

Provide a way to access elements of a collection sequentially without exposing its underlying structure.

Real-World Analogy

A TV remote with "next channel" and "previous channel" buttons. You don't need to know how channels are stored internally.

The Problem

A playlist stores songs. To iterate, the client must know the internal data structure (list, index-based). If the structure changes, the client breaks.

```
"""
================================================================
 BAD EXAMPLE: Without the Iterator Pattern
================================================================

A playlist that stores songs. To iterate, the client must
know the internal structure (a list, index-based access).

Problems:
    1. Client is tightly coupled to the internal data structure
    2. If you change from list to tree or database -> client breaks
    3. No way to have multiple independent traversals
    4. Client code is cluttered with indexing logic
================================================================
"""


class BadPlaylist:
    def __init__(self):
```

```
        self._songs = []

    def add(self, song: str):
        self._songs.append(song)

    def get_songs(self) -> list:
        return self._songs  # exposes internals!


if __name__ == "__main__":
    playlist = BadPlaylist()
    playlist.add("Bohemian Rhapsody")
    playlist.add("Hotel California")
    playlist.add("Stairway to Heaven")
    playlist.add("Imagine")

    # Client must know it's a list and use indexing
    songs = playlist.get_songs()
    for i in range(len(songs)):
        print(f"  {i + 1}. {songs[i]}")

    print()
    print("Problems:")
    print("  - Client accesses the raw list directly")
    print("  - What if we change to a linked list or database?")
    print("  - What if we want to iterate in reverse or shuffle?")
    print("  -> The Iterator pattern fixes this.")
```

The Solution

Provide iterator objects that traverse the collection without exposing its internals.

```
"""
================================================================
 DESIGN PATTERN #6: ITERATOR
================================================================

Category: Behavioral
Intent:   Provide a way to access elements of a collection
          sequentially without exposing its underlying structure.

Real-world analogy:
    A TV remote with "next channel" and "previous channel" buttons.
    You don't need to know how channels are stored internally —
    you just press next/previous.

Note: Python has built-in iterator support via __iter__ and __next__.
```

```
      This lesson shows BOTH the classic pattern AND the Pythonic way.

When to use:
    - You want to hide internal structure of a collection
    - You need multiple traversal strategies (forward, reverse, filtered)
    - You want a uniform interface for different collection types

============================================================
"""

from abc import ABC, abstractmethod
from typing import Any


# ------------------------------------------------------------
# STEP 1: Classic Iterator Pattern (language-agnostic)
# ------------------------------------------------------------

class Iterator(ABC):
    @abstractmethod
    def has_next(self) -> bool:
        pass

    @abstractmethod
    def next(self) -> Any:
        pass


class ForwardIterator(Iterator):
    def __init__(self, songs: list[str]):
        self._songs = songs
        self._index = 0

    def has_next(self) -> bool:
        return self._index < len(self._songs)

    def next(self) -> str:
        song = self._songs[self._index]
        self._index += 1
        return song


class ReverseIterator(Iterator):
    def __init__(self, songs: list[str]):
        self._songs = songs
        self._index = len(songs) - 1
```

```python
    def has_next(self) -> bool:
        return self._index >= 0

    def next(self) -> str:
        song = self._songs[self._index]
        self._index -= 1
        return song


class Playlist:
    def __init__(self):
        self._songs: list[str] = []

    def add(self, song: str) -> None:
        self._songs.append(song)

    def forward_iterator(self) -> ForwardIterator:
        return ForwardIterator(list(self._songs))

    def reverse_iterator(self) -> ReverseIterator:
        return ReverseIterator(list(self._songs))


# ------------------------------------------------------------
# STEP 2: Pythonic Iterator (using __iter__ and __next__)
# ------------------------------------------------------------
# Python's for-loop protocol uses these magic methods.

class PythonicPlaylist:
    def __init__(self):
        self._songs: list[str] = []

    def add(self, song: str) -> None:
        self._songs.append(song)

    def __iter__(self):
        """Makes this object iterable in a for-loop."""
        self._index = 0
        return self

    def __next__(self) -> str:
        if self._index >= len(self._songs):
            raise StopIteration
        song = self._songs[self._index]
        self._index += 1
        return song
```

```python
    def reverse(self):
        """A generator — the most Pythonic way to create iterators."""
        for song in reversed(self._songs):
            yield song


# ------------------------------------------------------------
# STEP 3: See it in action
# ------------------------------------------------------------

if __name__ == "__main__":
    # --- Classic Pattern ---
    print("=== Classic Iterator: Forward ===")
    playlist = Playlist()
    for song in ["Bohemian Rhapsody", "Hotel California",
                 "Stairway to Heaven", "Imagine"]:
        playlist.add(song)

    it = playlist.forward_iterator()
    i = 1
    while it.has_next():
        print(f"  {i}. {it.next()}")
        i += 1

    print("\n=== Classic Iterator: Reverse ===")
    it = playlist.reverse_iterator()
    i = 1
    while it.has_next():
        print(f"  {i}. {it.next()}")
        i += 1

    # --- Pythonic Way ---
    print("\n=== Pythonic Iterator: for-loop ===")
    py_playlist = PythonicPlaylist()
    for song in ["Bohemian Rhapsody", "Hotel California",
                 "Stairway to Heaven", "Imagine"]:
        py_playlist.add(song)

    for i, song in enumerate(py_playlist, 1):
        print(f"  {i}. {song}")

    print("\n=== Pythonic Iterator: Reverse (generator) ===")
    for i, song in enumerate(py_playlist.reverse(), 1):
        print(f"  {i}. {song}")

    print()
    print("=" * 60)
```

```
    print("KEY TAKEAWAYS:")
    print("=" * 60)
    print("1. Clients don't need to know the internal data structure.")
    print("2. Multiple traversal strategies (forward, reverse, etc.).")
    print("3. Python's __iter__/__next__ is the built-in Iterator pattern.")
    print("4. Generators (yield) are the most Pythonic iterators.")
    print("5. The pattern separates 'how to traverse' from 'what to store'.")
```

Structure

```
+------------+  creates   +------------------+
|  Playlist  | ----------> |    Iterator       | (interface)
| (Collection)|            |  has_next()       |
|            |            |  next()           |
+------------+            +------------------+
                              ^          ^
                              |          |
                     +------------+ +------------+
                     |  Forward   | |  Reverse   |
                     |  Iterator  | |  Iterator  |
                     +------------+ +------------+
```

Python's Built-in Support

- `__iter__()` + `__next__()` makes objects work with `for` loops
- `yield` (generators) is the most Pythonic way to create iterators

When to Use

- You want to hide the internal structure of a collection
- You need multiple traversal strategies
- You want a uniform interface for different collection types

Key Takeaways

1. Clients don't need to know the internal data structure
2. Multiple traversal strategies are easy to add
3. Python's `__iter__`/`__next__` IS the Iterator pattern
4. Generators (`yield`) are the most Pythonic iterators
5. Separates "how to traverse" from "what to store"

Chapter 7: Chain of Responsibility Pattern

Category: Behavioral

Intent

Pass a request along a chain of handlers. Each handler decides either to process the request or pass it to the next handler.

Real-World Analogy

A call center. You call tech support — they can't help — transferred to specialist — still unresolved — escalated to manager. Each person either handles it or passes it along.

The Problem

A support ticket system with levels: basic, manager, director. One giant method with nested if/elif for every level.

```python
"""
================================================================
 BAD EXAMPLE: Without the Chain of Responsibility Pattern
================================================================

A support ticket system with levels: basic, manager, director.

Problems:
    1. One giant method with nested if/elif for every level
    2. Adding a new support level -> modify the method
    3. Can't reorder or reconfigure the chain dynamically
    4. All logic crammed into one place
================================================================
"""


class BadSupportSystem:
    def handle_ticket(self, issue: str, severity: int):
        print(f"\n[TICKET] Ticket: '{issue}' (severity: {severity})")

        if severity <= 1:
            print(f"  [USER] Basic Support: Resolved '{issue}' with FAQ.")
        elif severity <= 3:
            print(f"  [MGR] Manager: Escalated and resolved '{issue}'.")
        elif severity <= 5:
            print(f"  [DIR] Director: Emergency meeting for '{issue}'.")
        else:
            print(f"  [x] No one can handle severity {severity}!")
        # Add VP level? CEO level? Edit this method again...


if __name__ == "__main__":
    system = BadSupportSystem()
```

```
    system.handle_ticket("Password reset", 1)
    system.handle_ticket("Billing dispute", 3)
    system.handle_ticket("Data breach", 5)
    system.handle_ticket("Alien invasion", 7)

    print()
    print("All logic is in one method with nested if/elif.")
    print("-> The Chain of Responsibility pattern fixes this.")
```

The Solution

Create a chain of handler objects. Each handler tries to process the request or passes it to the next.

```
"""
================================================================
 DESIGN PATTERN #7: CHAIN OF RESPONSIBILITY
================================================================

Category: Behavioral
Intent:   Pass a request along a chain of handlers. Each handler
          decides either to process the request or pass it to
          the next handler in the chain.

Real-world analogy:
    A call center. You call tech support -> if they can't help,
    they transfer you to a specialist -> if still unresolved,
    they escalate to a manager. Each person in the chain either
    handles it or passes it along.

When to use:
    - Multiple objects may handle a request, and the handler
      isn't known in advance
    - You want to decouple the sender from the receiver
    - You want to dynamically configure the processing chain

================================================================
"""

from abc import ABC, abstractmethod


# ------------------------------------------------------------
# STEP 1: The Handler interface
# ------------------------------------------------------------

class SupportHandler(ABC):
```

```python
    def __init__(self):
        self._next_handler: SupportHandler | None = None

    def set_next(self, handler: "SupportHandler") -> "SupportHandler":
        """Chain this handler to the next one. Returns next for fluent API."""
        self._next_handler = handler
        return handler

    def handle(self, issue: str, severity: int) -> None:
        """Try to handle, or pass to next in chain."""
        if self.can_handle(severity):
            self.process(issue, severity)
        elif self._next_handler:
            self._next_handler.handle(issue, severity)
        else:
            print(f"  [x] No handler available for '{issue}' (severity {severity})")

    @abstractmethod
    def can_handle(self, severity: int) -> bool:
        pass

    @abstractmethod
    def process(self, issue: str, severity: int) -> None:
        pass


# ---------------------------------------------------------
# STEP 2: Concrete handlers
# ---------------------------------------------------------

class BasicSupport(SupportHandler):
    def can_handle(self, severity: int) -> bool:
        return severity <= 1

    def process(self, issue: str, severity: int) -> None:
        print(f"  [USER] Basic Support: Resolved '{issue}' with FAQ.")


class ManagerSupport(SupportHandler):
    def can_handle(self, severity: int) -> bool:
        return severity <= 3

    def process(self, issue: str, severity: int) -> None:
        print(f"  [MGR] Manager: Escalated and resolved '{issue}'.")


class DirectorSupport(SupportHandler):
```

```python
    def can_handle(self, severity: int) -> bool:
        return severity <= 5

    def process(self, issue: str, severity: int) -> None:
        print(f"  [DIR] Director: Emergency meeting for '{issue}'.")


class CEOSupport(SupportHandler):
    """Easy to add a new level — just create a new class!"""
    def can_handle(self, severity: int) -> bool:
        return severity <= 10

    def process(self, issue: str, severity: int) -> None:
        print(f"  [CEO] CEO: Personal intervention for '{issue}'.")


# ------------------------------------------------------------
# STEP 3: See it in action
# ------------------------------------------------------------

if __name__ == "__main__":
    # Build the chain: Basic -> Manager -> Director -> CEO
    basic = BasicSupport()
    basic.set_next(ManagerSupport()) \
         .set_next(DirectorSupport()) \
         .set_next(CEOSupport())

    tickets = [
        ("Password reset", 1),
        ("Billing dispute", 3),
        ("Data breach", 5),
        ("Alien invasion", 7),
    ]

    for issue, severity in tickets:
        print(f"\n[TICKET] Ticket: '{issue}' (severity: {severity})")
        basic.handle(issue, severity)

    print()
    print("=" * 60)
    print("KEY TAKEAWAYS:")
    print("=" * 60)
    print("1. Each handler only knows about its own responsibility.")
    print("2. The chain is configurable — reorder or extend easily.")
    print("3. Adding CEO level = new class, no existing code changed.")
    print("4. The sender (ticket) doesn't know who will handle it.")
    print("5. If no one handles it, you get a clear fallback message.")
```

Structure

```
  Request
    |
    v
+---------+  next  +---------+  next  +----------+  next  +-----+
|  Basic  | ------> | Manager | ------> | Director | ------> | CEO |
| Support |        | Support |        | Support  |        |     |
+---------+        +---------+        +----------+        +-----+
```

When to Use

- Multiple objects may handle a request
- The handler isn't known in advance
- You want to dynamically configure the chain

Key Takeaways

1. Each handler only knows its own responsibility
2. The chain is configurable and extensible
3. Sender doesn't know who will handle its request
4. Follows Single Responsibility Principle

Chapter 8: Mediator Pattern

Category: Behavioral

Intent

Define an object that encapsulates how a set of objects interact. Promotes loose coupling by keeping objects from referring to each other explicitly.

Real-World Analogy

An air traffic control tower. Planes don't communicate directly — they all talk to the tower, which coordinates everything.

The Problem

A chat room where users communicate directly with each other. Every user must hold references to every other user. N users = N*(N-1) connections.

```
"""
================================================================
 BAD EXAMPLE: Without the Mediator Pattern
================================================================
```

```
A chat room where users communicate directly with each other.

Problems:
    1. Every user must hold references to every other user
    2. Adding a new user -> update ALL existing users
    3. N users = N*(N-1) connections (exponential complexity)
    4. Can't add features like logging or filtering centrally
================================================================
"""


class BadUser:
    def __init__(self, name: str):
        self.name = name
        self.contacts: list["BadUser"] = []

    def add_contact(self, user: "BadUser"):
        self.contacts.append(user)

    def send(self, message: str):
        print(f"  {self.name} sends: '{message}'")
        for contact in self.contacts:
            contact.receive(message, self.name)

    def receive(self, message: str, sender: str):
        print(f"    {self.name} received from {sender}: '{message}'")


if __name__ == "__main__":
    alice = BadUser("Alice")
    bob = BadUser("Bob")
    charlie = BadUser("Charlie")

    # Every user must know about every other user!
    alice.add_contact(bob)
    alice.add_contact(charlie)
    bob.add_contact(alice)
    bob.add_contact(charlie)
    charlie.add_contact(alice)
    charlie.add_contact(bob)

    alice.send("Hello everyone!")

    print()
    print("3 users = 6 connections. 10 users = 90 connections!")
    print("-> The Mediator pattern fixes this.")
```

The Solution

Route all communication through a central mediator. Users only know the mediator, not each other.

```
"""
================================================================
 DESIGN PATTERN #8: MEDIATOR
================================================================

Category: Behavioral
Intent:   Define an object that encapsulates how a set of objects
          interact. Promotes loose coupling by keeping objects from
          referring to each other explicitly.

Real-world analogy:
    An air traffic control tower. Planes don't communicate directly
    with each other — they all talk to the tower, and the tower
    coordinates everything. Without the tower, every plane would
    need to know about every other plane.

When to use:
    - Many objects communicate in complex ways
    - You want to centralize control logic
    - Objects should not refer to each other directly

================================================================
"""

from abc import ABC, abstractmethod


# ------------------------------------------------------------
# STEP 1: The Mediator interface
# ------------------------------------------------------------

class ChatMediator(ABC):
    @abstractmethod
    def send_message(self, message: str, sender: "User") -> None:
        pass

    @abstractmethod
    def add_user(self, user: "User") -> None:
        pass


# ------------------------------------------------------------
# STEP 2: Concrete Mediator — the "control tower"
# ------------------------------------------------------------
```

```python
class ChatRoom(ChatMediator):
    def __init__(self, name: str):
        self.name = name
        self._users: list["User"] = []

    def add_user(self, user: "User") -> None:
        self._users.append(user)
        user.chat_room = self
        print(f"  [v] {user.name} joined #{self.name}")

    def send_message(self, message: str, sender: "User") -> None:
        print(f"  [CHAT] [{self.name}] {sender.name}: '{message}'")
        for user in self._users:
            if user != sender:
                user.receive(message, sender.name)


# ------------------------------------------------------------
# STEP 3: The Colleague — each user only knows the mediator
# ------------------------------------------------------------

class User:
    def __init__(self, name: str):
        self.name = name
        self.chat_room: ChatMediator | None = None

    def send(self, message: str) -> None:
        if self.chat_room:
            self.chat_room.send_message(message, self)

    def receive(self, message: str, sender: str) -> None:
        print(f"    [MSG] {self.name} received from {sender}: '{message}'")


# ------------------------------------------------------------
# STEP 4: See it in action
# ------------------------------------------------------------

if __name__ == "__main__":
    # Create the mediator (chat room)
    room = ChatRoom("general")

    # Create users — they DON'T know about each other
    alice = User("Alice")
    bob = User("Bob")
    charlie = User("Charlie")
```

```python
    # Add users to the room
    print("=== Users joining ===")
    room.add_user(alice)
    room.add_user(bob)
    room.add_user(charlie)

    # Communication goes through the mediator
    print("\n=== Chatting ===")
    alice.send("Hello everyone!")
    bob.send("Hey Alice!")

    # Easy to add a new user - no changes to existing users
    print("\n=== New user joins ===")
    diana = User("Diana")
    room.add_user(diana)
    diana.send("Hi, I'm new here!")

    print()
    print("=" * 60)
    print("KEY TAKEAWAYS:")
    print("=" * 60)
    print("1. Users don't know about each other - only the ChatRoom.")
    print("2. 10 users = 10 connections (to mediator), not 90.")
    print("3. Central place to add logging, filtering, etc.")
    print("4. New user joins? Just register with the mediator.")
    print("5. Similar to Observer, but Mediator is bidirectional")
    print("   and coordinates complex interactions.")
```

Structure

```
        +---------------+
        |   ChatRoom    |
        |  (Mediator)   |
        |               |
        | send_message() |
        +---------------+
         ^    ^    ^
         |    |    |       All communication
         |    |    |       goes through
         |    |    |       the mediator
      +--+    |    +--+
      |       |       |
  +-------+ +-----+ +---------+
  | Alice | | Bob | | Charlie |
  +-------+ +-----+ +---------+
      (Users only know the mediator)
```

Mediator vs Observer

	Observer	Mediator
Direction	One-way broadcast	Bidirectional coordination
Awareness	Observers don't know each other	Colleagues don't know each other
Complexity	Simple notification	Complex interaction logic

When to Use

- Many objects communicate in complex ways
- You want to centralize control logic
- Objects should not refer to each other directly

Key Takeaways

1. Colleagues only know the mediator, not each other
2. N objects = N connections instead of N*(N-1)
3. Central place for logging, filtering, validation
4. Easy to add new participants

Chapter 9: Memento Pattern

Category: Behavioral

Intent

Capture and externalize an object's internal state so it can be restored later, without violating encapsulation.

Real-World Analogy

A save point in a video game. Save before the boss fight, die, load the save, try again. The save captures your exact state.

The Problem

A game character whose state you want to save and restore. To save state, you must expose ALL internal fields.

```
"""
================================================================
 BAD EXAMPLE: Without the Memento Pattern
================================================================

A game character whose state you want to save and restore.
```

```
Problems:
    1. To save state, you must expose ALL internal fields
    2. Other code can modify the saved state (no encapsulation)
    3. If you add a new field, you must update save/restore everywhere
    4. No structured history — just raw data floating around
==============================================================
"""


class BadGameCharacter:
    def __init__(self, name: str):
        self.name = name
        self.level = 1
        self.hp = 100
        self.position = (0, 0)

    def take_damage(self, dmg: int):
        self.hp -= dmg
        print(f"  {self.name} took {dmg} damage. HP: {self.hp}")

    def level_up(self):
        self.level += 1
        self.hp = 100
        print(f"  {self.name} leveled up to {self.level}! HP restored.")

    def move(self, x: int, y: int):
        self.position = (x, y)
        print(f"  {self.name} moved to {self.position}")


if __name__ == "__main__":
    hero = BadGameCharacter("Hero")
    hero.move(5, 3)
    hero.take_damage(40)

    # "Save" by manually copying fields — fragile!
    saved_level = hero.level
    saved_hp = hero.hp
    saved_pos = hero.position

    hero.take_damage(60)
    print(f"\n   {hero.name} died! HP: {hero.hp}")

    # "Restore" by manually setting fields
    hero.level = saved_level
    hero.hp = saved_hp
    hero.position = saved_pos
```

```
    print(f"  Restored! Level: {hero.level}, HP: {hero.hp}, Pos: {hero.position}")

    print()
    print("This works, but it exposes internal state to outside code.")
    print("Add a new field (e.g., inventory)? Must update save/restore.")
    print("-> The Memento pattern fixes this.")
```

The Solution

Create immutable snapshot objects (mementos) that capture state. A caretaker manages the history of snapshots.

```
"""
================================================================
 DESIGN PATTERN #9: MEMENTO
================================================================

Category: Behavioral
Intent:   Capture and externalize an object's internal state
          so that it can be restored later, without violating
          encapsulation.

Real-world analogy:
    A save point in a video game. You save your progress before
    a boss fight. If you die, you load the save and try again.
    The save file captures your exact state without exposing
    the game's internal code.

Key terms:
    - Originator: the object whose state is saved (GameCharacter)
    - Memento:    the snapshot object (immutable state capsule)
    - Caretaker:  manages the history of mementos (SaveManager)

When to use:
    - You need save/restore (undo, checkpoints, snapshots)
    - You want to preserve encapsulation of internal state
    - You need a history of states

================================================================
"""

from dataclasses import dataclass
from copy import deepcopy


# ------------------------------------------------------------
# STEP 1: The Memento — an immutable snapshot
```

```python
# ------------------------------------------------------------
# Stores the state but doesn't expose setters.

@dataclass(frozen=True)  # frozen = immutable!
class GameMemento:
    level: int
    hp: int
    position: tuple
    inventory: list  # new field — no problem!
    label: str = ""


# ------------------------------------------------------------
# STEP 2: The Originator — creates and restores from mementos
# ------------------------------------------------------------

class GameCharacter:
    def __init__(self, name: str):
        self.name = name
        self.level = 1
        self.hp = 100
        self.position = (0, 0)
        self.inventory: list[str] = []

    def take_damage(self, dmg: int) -> None:
        self.hp = max(0, self.hp - dmg)
        print(f"  [DMG]  {self.name} took {dmg} damage. HP: {self.hp}")

    def level_up(self) -> None:
        self.level += 1
        self.hp = 100
        print(f"  [UP]  {self.name} leveled up to {self.level}! HP restored.")

    def move(self, x: int, y: int) -> None:
        self.position = (x, y)
        print(f"  [WALK] {self.name} moved to {self.position}")

    def pick_up(self, item: str) -> None:
        self.inventory.append(item)
        print(f"  [BAG] {self.name} picked up '{item}'")

    def save(self, label: str = "") -> GameMemento:
        """Create a memento (snapshot) of current state."""
        return GameMemento(
            level=self.level,
            hp=self.hp,
            position=self.position,
```

```
            inventory=deepcopy(self.inventory),
            label=label,
        )

    def restore(self, memento: GameMemento) -> None:
        """Restore state from a memento."""
        self.level = memento.level
        self.hp = memento.hp
        self.position = memento.position
        self.inventory = deepcopy(memento.inventory)
        print(f"  [SAVE] Restored to '{memento.label}': "
              f"Level {self.level}, HP {self.hp}, Pos {self.position}, "
              f"Items {self.inventory}")

    def status(self) -> str:
        return (f"  [STAT] {self.name}: Level {self.level}, HP {self.hp}, "
                f"Pos {self.position}, Items {self.inventory}")


# ------------------------------------------------------------
# STEP 3: The Caretaker — manages save history
# ------------------------------------------------------------

class SaveManager:
    def __init__(self):
        self._saves: list[GameMemento] = []

    def save(self, memento: GameMemento) -> None:
        self._saves.append(memento)
        print(f"  [SAVE] Game saved: '{memento.label}'")

    def load(self, index: int = -1) -> GameMemento:
        return self._saves[index]

    def list_saves(self) -> None:
        print("  [DIR] Save files:")
        for i, m in enumerate(self._saves):
            print(f"     [{i}] '{m.label}' - Level {m.level}, HP {m.hp}")


# ------------------------------------------------------------
# STEP 4: See it in action
# ------------------------------------------------------------

if __name__ == "__main__":
    hero = GameCharacter("Hero")
    saves = SaveManager()
```

```
    print("=== Adventure begins ===")
    hero.move(5, 3)
    hero.pick_up("Sword")
    hero.level_up()

    # Save before boss fight
    saves.save(hero.save("Before boss fight"))

    print("\n=== Boss fight (goes badly) ===")
    hero.take_damage(40)
    hero.take_damage(35)
    hero.pick_up("Poison")
    hero.take_damage(30)
    print(hero.status())

    print("\n=== Load save! ===")
    saves.list_saves()
    hero.restore(saves.load(0))
    print(hero.status())

    print("\n=== Boss fight (retry, goes well) ===")
    hero.take_damage(20)
    hero.pick_up("Boss Key")
    hero.level_up()
    saves.save(hero.save("After boss defeated"))

    print("\n=== All saves ===")
    saves.list_saves()

    print()
    print("=" * 60)
    print("KEY TAKEAWAYS:")
    print("=" * 60)
    print("1. Internal state is captured WITHOUT exposing it.")
    print("2. Mementos are immutable — can't be tampered with.")
    print("3. The Caretaker manages saves without knowing the details.")
    print("4. Adding new fields (inventory) doesn't break anything.")
    print("5. Different from Command: Command stores actions,")
    print("   Memento stores snapshots of state.")
```

Structure

```
+--------------+  creates   +-------------+  stores   +-------------+
|GameCharacter | ----------> | GameMemento | <--------- | SaveManager |
| (Originator) |            | (Snapshot)  |           | (Caretaker) |
|              | <---------- |  level      |           |             |
|  save()      |  restores  |  hp         |           | saves: []   |
```

```
|  restore()   |            |  position   |           |             |
+--------------+            +-------------+           +-------------+
```

Command vs Memento

	Command	Memento
Stores	Actions (do/undo)	State snapshots
Undo approach	Reverse the action	Restore full state
Best for	Action history	Checkpoint/rollback

When to Use

- You need save/restore (undo, checkpoints, snapshots)
- You want to preserve encapsulation
- You need a history of states

Key Takeaways

1. Internal state captured without exposing it
2. Mementos are immutable
3. Caretaker manages saves without knowing details
4. Adding new fields doesn't break existing code

Chapter 10: Visitor Pattern

Category: Behavioral

Intent

Add new operations to existing object structures without modifying those objects. Separate algorithms from the objects they operate on.

Real-World Analogy

A tax inspector visiting businesses. The inspector applies different tax rules to each business type, but businesses don't need to know about tax rules — they just "accept" the inspection.

The Problem

A document with different element types (Text, Image, Table). Adding a new operation (HTML export, PDF export) means modifying EVERY element class.

```python
"""
================================================================
 BAD EXAMPLE: Without the Visitor Pattern
================================================================

A document with different element types (Text, Image, Table).
You need to add operations: export to HTML, calculate size, etc.

Problems:
    1. Adding a new operation = modify EVERY element class
    2. Element classes get bloated with unrelated methods
    3. Violates Single Responsibility — elements shouldn't know
       about HTML export, PDF export, size calculation, etc.
================================================================
"""


class BadText:
    def __init__(self, content: str):
        self.content = content

    def to_html(self):
        return f"<p>{self.content}</p>"

    def calculate_size(self):
        return len(self.content)

    # to_pdf()? to_markdown()? to_latex()?
    # Each new operation = modify THIS class!


class BadImage:
    def __init__(self, url: str, width: int, height: int):
        self.url = url
        self.width = width
        self.height = height

    def to_html(self):
        return f'<img src="{self.url}" width="{self.width}" height="{self.height}">'

    def calculate_size(self):
        return self.width * self.height

    # Same problem — every new operation goes here


if __name__ == "__main__":
```

```
    elements = [BadText("Hello World"), BadImage("cat.jpg", 800, 600)]

    print("=== HTML Export ===")
    for elem in elements:
        print(f"  {elem.to_html()}")

    print("\n=== Size Calculation ===")
    for elem in elements:
        print(f"  Size: {elem.calculate_size()}")

    print()
    print("Every new operation (PDF export, markdown, etc.)")
    print("requires modifying ALL element classes.")
    print("-> The Visitor pattern fixes this.")
```

The Solution

Elements accept visitors. Each visitor contains the logic for one operation across all element types.

```
"""
================================================================
 DESIGN PATTERN #10: VISITOR
================================================================

Category: Behavioral
Intent:   Add new operations to existing object structures without
          modifying those objects. Separate algorithms from the
          objects they operate on.

Real-world analogy:
    A tax inspector visiting different businesses. The inspector
    (visitor) applies a different tax calculation to each type
    of business (restaurant, shop, office), but the businesses
    don't need to know about tax rules — they just "accept"
    the inspector's visit.

When to use:
    - You need to add operations to classes without modifying them
    - You have a stable class hierarchy but frequently add operations
    - Related operations should be grouped together (not scattered)

================================================================
"""

from abc import ABC, abstractmethod
```

```python
# ------------------------------------------------------------
# STEP 1: Element interface — just needs accept()
# ------------------------------------------------------------

class DocumentElement(ABC):
    @abstractmethod
    def accept(self, visitor: "DocumentVisitor") -> str:
        pass


# ------------------------------------------------------------
# STEP 2: Concrete elements — stable, rarely change
# ------------------------------------------------------------

class Text(DocumentElement):
    def __init__(self, content: str):
        self.content = content

    def accept(self, visitor: "DocumentVisitor") -> str:
        return visitor.visit_text(self)


class Image(DocumentElement):
    def __init__(self, url: str, width: int, height: int):
        self.url = url
        self.width = width
        self.height = height

    def accept(self, visitor: "DocumentVisitor") -> str:
        return visitor.visit_image(self)


class Table(DocumentElement):
    def __init__(self, rows: int, cols: int):
        self.rows = rows
        self.cols = cols

    def accept(self, visitor: "DocumentVisitor") -> str:
        return visitor.visit_table(self)


# ------------------------------------------------------------
# STEP 3: Visitor interface
# ------------------------------------------------------------

class DocumentVisitor(ABC):
    @abstractmethod
```

```
    def visit_text(self, element: Text) -> str:
        pass

    @abstractmethod
    def visit_image(self, element: Image) -> str:
        pass

    @abstractmethod
    def visit_table(self, element: Table) -> str:
        pass


# ---------------------------------------------------------
# STEP 4: Concrete visitors — each is a complete operation
# ---------------------------------------------------------
# Adding a new operation = just add a new Visitor class!

class HtmlExportVisitor(DocumentVisitor):
    def visit_text(self, element: Text) -> str:
        return f"<p>{element.content}</p>"

    def visit_image(self, element: Image) -> str:
        return f'<img src="{element.url}" width="{element.width}" height="{element.heig

    def visit_table(self, element: Table) -> str:
        rows = "".join("<tr>" + "<td></td>" * element.cols + "</tr>"
                       for _ in range(element.rows))
        return f"<table>{rows}</table>"


class MarkdownExportVisitor(DocumentVisitor):
    """New operation — no changes to Text, Image, or Table!"""
    def visit_text(self, element: Text) -> str:
        return element.content

    def visit_image(self, element: Image) -> str:
        return f"![image]({element.url})"

    def visit_table(self, element: Table) -> str:
        header = "| " + " | ".join(f"Col{i+1}" for i in range(element.cols)) + " |"
        separator = "| " + " | ".join("---" for _ in range(element.cols)) + " |"
        row = "| " + " | ".join("..." for _ in range(element.cols)) + " |"
        rows = "\n".join([header, separator] + [row] * element.rows)
        return rows


class SizeCalculatorVisitor(DocumentVisitor):
```

```python
    """Another operation — still no changes to element classes!"""
    def visit_text(self, element: Text) -> str:
        size = len(element.content)
        return f"Text: {size} chars"

    def visit_image(self, element: Image) -> str:
        size = element.width * element.height
        return f"Image: {size} pixels"

    def visit_table(self, element: Table) -> str:
        size = element.rows * element.cols
        return f"Table: {size} cells"


# ------------------------------------------------------------
# STEP 5: See it in action
# ------------------------------------------------------------

if __name__ == "__main__":
    # Create document elements
    document = [
        Text("Hello World"),
        Image("cat.jpg", 800, 600),
        Table(3, 4),
    ]

    # Apply different visitors to the SAME elements
    print("=== HTML Export ===")
    html_visitor = HtmlExportVisitor()
    for elem in document:
        print(f"  {elem.accept(html_visitor)}")

    print("\n=== Markdown Export ===")
    md_visitor = MarkdownExportVisitor()
    for elem in document:
        print(f"  {elem.accept(md_visitor)}")

    print("\n=== Size Calculator ===")
    size_visitor = SizeCalculatorVisitor()
    for elem in document:
        print(f"  {elem.accept(size_visitor)}")

    print()
    print("=" * 60)
    print("KEY TAKEAWAYS:")
    print("=" * 60)
    print("1. Element classes (Text, Image, Table) were NOT modified.")
```

```
    print("2. Three operations added by creating three Visitor classes.")
    print("3. Each visitor groups related logic for ALL element types.")
    print("4. Trade-off: adding a new ELEMENT type requires updating")
    print("   all visitors. Best when elements are stable but")
    print("   operations change frequently.")
```

Structure

```
+----------------+  accepts   +-------------------+
|DocumentElement | ----------> | DocumentVisitor   |
|  accept()      |            |  visit_text()     |
+----------------+            |  visit_image()    |
    ^    ^    ^               |  visit_table()    |
    |    |    |               +-------------------+
 +--+    |    +--+               ^       ^
 |       |       |               |       |
Text   Image   Table    HtmlVisitor  MarkdownVisitor
```

The Double Dispatch Trick

1. Client calls `element.accept(visitor)`
2. Element calls `visitor.visit_text(self)` (or visit_image, etc.)
3. The correct method is called based on BOTH the element type AND visitor type

Trade-off

- Easy to add new **operations** (new Visitor class)
- Hard to add new **element types** (must update all Visitors)
- Best when: elements are stable, operations change frequently

When to Use

- You need to add operations without modifying classes
- You have a stable class hierarchy
- Related operations should be grouped together

Key Takeaways

1. Element classes are NOT modified when adding operations
2. Each visitor groups related logic for all element types
3. Uses “double dispatch” to pick the right method
4. Trade-off: new elements require updating all visitors

Part II: Creational Patterns

Chapter 11: Factory Method Pattern

Category: Creational

Intent

Define an interface for creating an object, but let subclasses decide which class to instantiate.

Real-World Analogy

A hiring agency. You say "I need a developer" and the agency decides which specific developer to send. Different agencies specialize in different types.

The Problem

A logistics app that creates different transports. Client code is tightly coupled to concrete classes.

```
"""
================================================================
 BAD EXAMPLE: Without the Factory Method Pattern
================================================================

A logistics app that creates different transports.

Problems:
    1. Client code is tightly coupled to concrete classes
    2. Adding a new transport = modify client code everywhere
    3. Object creation logic is scattered across the codebase
    4. Hard to test — can't substitute mock objects easily
================================================================
"""
```

```python
class Truck:
    def deliver(self):
        return "[PKG] Delivering by truck on the road"


class Ship:
    def deliver(self):
        return " Delivering by ship across the sea"


def create_transport(transport_type: str):
    """Creation logic scattered and coupled to concrete types."""
    if transport_type == "truck":
        return Truck()
    elif transport_type == "ship":
        return Ship()
    # elif transport_type == "drone": ...  <- modify HERE
    # elif transport_type == "train": ...  <- and HERE
    else:
        raise ValueError(f"Unknown transport: {transport_type}")


if __name__ == "__main__":
    for t in ["truck", "ship"]:
        transport = create_transport(t)
        print(f"  {transport.deliver()}")

    print()
    print("Client must know about 'truck', 'ship' as strings.")
    print("Adding 'drone' means changing the factory function")
    print("AND any code that calls it.")
    print("-> The Factory Method pattern fixes this.")
```

The Solution

Let subclasses decide which product to create via a factory method.

```python
"""
============================================================
 DESIGN PATTERN #11: FACTORY METHOD
============================================================

Category: Creational
Intent:   Define an interface for creating an object, but let
          subclasses decide which class to instantiate.

Real-world analogy:
```

```
    A hiring agency. You tell the agency "I need a developer"
    (the what), and the agency decides which specific developer
    to send (the how). Different agencies specialize in different
    types of developers.

When to use:
    - You don't know in advance which class to instantiate
    - You want subclasses to control object creation
    - You want to decouple creation from usage

================================================================
"""

from abc import ABC, abstractmethod


# ----------------------------------------------------------
# STEP 1: Product interface
# ----------------------------------------------------------

class Transport(ABC):
    @abstractmethod
    def deliver(self) -> str:
        pass

    @abstractmethod
    def get_cost_per_km(self) -> float:
        pass


# ----------------------------------------------------------
# STEP 2: Concrete products
# ----------------------------------------------------------

class Truck(Transport):
    def deliver(self) -> str:
        return "[PKG] Delivering by truck on the road"

    def get_cost_per_km(self) -> float:
        return 1.5


class Ship(Transport):
    def deliver(self) -> str:
        return " Delivering by ship across the sea"

    def get_cost_per_km(self) -> float:
```

```python
        return 0.5


class Drone(Transport):
    def deliver(self) -> str:
        return "[DRONE] Delivering by drone through the air"

    def get_cost_per_km(self) -> float:
        return 5.0


# ---------------------------------------------------------
# STEP 3: Creator with the factory method
# ---------------------------------------------------------

class Logistics(ABC):
    @abstractmethod
    def create_transport(self) -> Transport:
        """The factory method — subclasses decide WHAT to create."""
        pass

    def plan_delivery(self, distance: int) -> None:
        """Uses the factory method — doesn't know the concrete type."""
        transport = self.create_transport()
        cost = transport.get_cost_per_km() * distance
        print(f"  {transport.deliver()}")
        print(f"    Distance: {distance}km, Cost: ${cost:.2f}")


# ---------------------------------------------------------
# STEP 4: Concrete creators
# ---------------------------------------------------------

class RoadLogistics(Logistics):
    def create_transport(self) -> Transport:
        return Truck()


class SeaLogistics(Logistics):
    def create_transport(self) -> Transport:
        return Ship()


class AirLogistics(Logistics):
    """New logistics type — no existing code changed!"""
    def create_transport(self) -> Transport:
        return Drone()
```

```python
# ----------------------------------------------------------
# STEP 5: See it in action
# ----------------------------------------------------------

if __name__ == "__main__":
    logistics_options: list[Logistics] = [
        RoadLogistics(),
        SeaLogistics(),
        AirLogistics(),
    ]

    for logistics in logistics_options:
        print(f"=== {logistics.__class__.__name__} ===")
        logistics.plan_delivery(100)
        print()

    print("=" * 60)
    print("KEY TAKEAWAYS:")
    print("=" * 60)
    print("1. plan_delivery() doesn't know if it's a Truck or Drone.")
    print("2. Each creator subclass decides which product to make.")
    print("3. Adding AirLogistics = new class, zero changes elsewhere.")
    print("4. The 'factory method' is create_transport() — that's it.")
    print("5. Client code works with the Logistics interface,")
    print("   never with concrete classes directly.")
```

Structure

```
+------------------+         +---------------+
|    Logistics     | creates |   Transport   |
|    (Creator)     | -------> |   (Product)   |
|                  |         |   deliver()   |
| create_transport()|         +---------------+
| plan_delivery()  |            ^    ^    ^
+------------------+            |    |    |
     ^    ^    ^          Truck  Ship  Drone
     |    |    |
  Road  Sea  Air
  Log.  Log. Log.
```

When to Use

- You don't know in advance which class to instantiate
- You want subclasses to control object creation
- You want to decouple creation from usage

Key Takeaways

1. Client works with interfaces, not concrete classes
2. Each creator subclass decides which product to create
3. Adding new types = new classes, no changes to existing code
4. The factory method is just one method — `create_transport()`
5. Follows Open/Closed Principle

Chapter 12: Abstract Factory Pattern

Category: Creational

Intent

Provide an interface for creating families of related objects without specifying their concrete classes.

Real-World Analogy

A furniture catalog: "Modern", "Victorian". Pick a catalog and every piece (chair, sofa, table) is guaranteed to match.

The Problem

A UI toolkit that supports Windows and Mac styles. Client must manually coordinate which button + checkbox to use. Easy to mix families.

```
"""
================================================================
 BAD EXAMPLE: Without the Abstract Factory Pattern
================================================================

A UI toolkit that supports Windows and Mac styles.

Problems:
    1. Client must manually coordinate which button + checkbox to use
    2. Easy to mix Windows button with Mac checkbox (inconsistency)
    3. Adding a new OS theme = scattered changes everywhere
================================================================
"""



class WindowsButton:
    def render(self): return "[Windows Button]"

class WindowsCheckbox:
```

```
    def render(self): return "[Windows ☑]"

class MacButton:
    def render(self): return "[Mac Button]"

class MacCheckbox:
    def render(self): return "[Mac ☑]"


def create_ui(os_type: str):
    if os_type == "windows":
        button = WindowsButton()
        checkbox = WindowsCheckbox()
    elif os_type == "mac":
        button = MacButton()
        checkbox = MacCheckbox()
    else:
        raise ValueError(f"Unknown OS: {os_type}")

    print(f"  {button.render()} {checkbox.render()}")


if __name__ == "__main__":
    create_ui("windows")
    create_ui("mac")

    print()
    print("What if someone accidentally mixes WindowsButton + MacCheckbox?")
    mixed_button = WindowsButton()
    mixed_checkbox = MacCheckbox()
    print(f"  Inconsistent: {mixed_button.render()} {mixed_checkbox.render()}")
    print("-> The Abstract Factory pattern prevents this.")
```

The Solution

Each factory creates a consistent family of products. The client works only with factory and product interfaces.

```
"""
================================================================
 DESIGN PATTERN #12: ABSTRACT FACTORY
================================================================

Category: Creational
Intent:   Provide an interface for creating FAMILIES of related
          objects without specifying their concrete classes.
```

```
Real-world analogy:
    A furniture store with catalogs: "Modern", "Victorian", "Art Deco".
    Each catalog has a matching chair, sofa, and table. You pick a
    catalog (factory) and everything you get is guaranteed to match.

Difference from Factory Method:
    - Factory Method: creates ONE product via subclassing
    - Abstract Factory: creates a FAMILY of related products

When to use:
    - You need to create families of related objects
    - You want to ensure consistency within a family
    - You want to swap entire families at once

==============================================================
"""

from abc import ABC, abstractmethod


# ----------------------------------------------------------
# STEP 1: Product interfaces (what each product must do)
# ----------------------------------------------------------

class Button(ABC):
    @abstractmethod
    def render(self) -> str:
        pass

class Checkbox(ABC):
    @abstractmethod
    def render(self) -> str:
        pass

class TextField(ABC):
    @abstractmethod
    def render(self) -> str:
        pass


# ----------------------------------------------------------
# STEP 2: Concrete products — Windows family
# ----------------------------------------------------------

class WindowsButton(Button):
    def render(self) -> str:
        return "[ Windows Button ]"
```

```python
class WindowsCheckbox(Checkbox):
    def render(self) -> str:
        return "[ Windows ☑ ]"

class WindowsTextField(TextField):
    def render(self) -> str:
        return "[ Windows _____ ]"


# ----------------------------------------------------------
# STEP 3: Concrete products — Mac family
# ----------------------------------------------------------

class MacButton(Button):
    def render(self) -> str:
        return "( Mac Button )"

class MacCheckbox(Checkbox):
    def render(self) -> str:
        return "( Mac ☑ )"

class MacTextField(TextField):
    def render(self) -> str:
        return "( Mac _____ )"


# ----------------------------------------------------------
# STEP 4: The Abstract Factory interface
# ----------------------------------------------------------

class UIFactory(ABC):
    @abstractmethod
    def create_button(self) -> Button:
        pass

    @abstractmethod
    def create_checkbox(self) -> Checkbox:
        pass

    @abstractmethod
    def create_text_field(self) -> TextField:
        pass


# ----------------------------------------------------------
# STEP 5: Concrete factories — each produces a FAMILY
# ----------------------------------------------------------
```

```python
class WindowsFactory(UIFactory):
    def create_button(self) -> Button:
        return WindowsButton()

    def create_checkbox(self) -> Checkbox:
        return WindowsCheckbox()

    def create_text_field(self) -> TextField:
        return WindowsTextField()


class MacFactory(UIFactory):
    def create_button(self) -> Button:
        return MacButton()

    def create_checkbox(self) -> Checkbox:
        return MacCheckbox()

    def create_text_field(self) -> TextField:
        return MacTextField()


# ---------------------------------------------------------
# STEP 6: Client code — works with ANY factory
# ---------------------------------------------------------

def render_ui(factory: UIFactory) -> None:
    """This function has NO idea if it's Windows or Mac."""
    button = factory.create_button()
    checkbox = factory.create_checkbox()
    text_field = factory.create_text_field()

    print(f"  {button.render()}")
    print(f"  {checkbox.render()}")
    print(f"  {text_field.render()}")


# ---------------------------------------------------------
# STEP 7: See it in action
# ---------------------------------------------------------

if __name__ == "__main__":
    print("=== Windows UI ===")
    render_ui(WindowsFactory())

    print("\n=== Mac UI ===")
    render_ui(MacFactory())
```

```
    print()
    print("=" * 60)
    print("KEY TAKEAWAYS:")
    print("=" * 60)
    print("1. render_ui() doesn't know Windows vs Mac.")
    print("2. Each factory guarantees a CONSISTENT family.")
    print("3. Can't accidentally mix Windows button + Mac checkbox.")
    print("4. Swap the entire theme by changing ONE factory object.")
    print("5. Factory Method = one product, Abstract Factory = family.")
```

Structure

```
+---------------+  creates  +--------+ +----------+ +-----------+
|  UIFactory    | ---------> | Button | | Checkbox | | TextField |
+---------------+           +--------+ +----------+ +-----------+
    ^        ^
    |        |
+--------+ +--------+
|Windows | |  Mac   |   Each factory creates a
|Factory | |Factory |   CONSISTENT family
+--------+ +--------+
```

Factory Method vs Abstract Factory

	Factory Method	Abstract Factory
Creates	One product	Family of products
Mechanism	Subclass overrides one method	Object with multiple create methods
Consistency	N/A	Guarantees matching family

Key Takeaways

1. Client doesn't know concrete classes
2. Each factory guarantees a consistent family
3. Can't mix products from different families
4. Swap entire theme by changing one factory

Chapter 13: Builder Pattern

Category: Creational

Intent

Separate the construction of a complex object from its representation, allowing the same process to create different representations.

Real-World Analogy

Ordering a custom pizza. You don't get a constructor with 20 parameters. Instead: dough, sauce, cheese, toppings, step by step.

The Problem

Creating a complex object (a House) with many optional parameters. A constructor with 10 parameters is unreadable.

```
"""
==============================================================
 BAD EXAMPLE: Without the Builder Pattern
==============================================================

Creating a complex object (a House) with many optional parameters.

Problems:
    1. Constructor with too many parameters (telescoping constructor)
    2. Hard to remember parameter order
    3. Many parameters are optional -> lots of None defaults
    4. No way to enforce valid combinations
==============================================================
"""


class BadHouse:
    def __init__(self, floors=1, rooms=1, has_garage=False,
                 has_pool=False, has_garden=False, has_solar=False,
                 wall_material="brick", roof_type="flat",
                 has_security=False, has_smart_home=False):
        self.floors = floors
        self.rooms = rooms
        self.has_garage = has_garage
        self.has_pool = has_pool
        self.has_garden = has_garden
        self.has_solar = has_solar
        self.wall_material = wall_material
        self.roof_type = roof_type
        self.has_security = has_security
        self.has_smart_home = has_smart_home

    def __str__(self):
        features = []
```

```
        if self.has_garage: features.append("garage")
        if self.has_pool: features.append("pool")
        if self.has_garden: features.append("garden")
        if self.has_solar: features.append("solar")
        if self.has_security: features.append("security")
        if self.has_smart_home: features.append("smart home")
        return (f"House: {self.floors}F, {self.rooms}R, "
                f"{self.wall_material} walls, {self.roof_type} roof, "
                f"features: {features or 'none'}")


if __name__ == "__main__":
    # Which parameter is which?? This is unreadable!
    house = BadHouse(2, 5, True, True, False, True, "wood", "gable", True, False)
    print(f"  {house}")

    print()
    print("10 parameters in the constructor!")
    print("What does the 4th True mean? Nobody knows without checking.")
    print("-> The Builder pattern fixes this.")
```

The Solution

Build the object step by step with named methods and method chaining.

```
"""
==============================================================
 DESIGN PATTERN #13: BUILDER
==============================================================

Category: Creational
Intent:   Separate the construction of a complex object from its
          representation, allowing the same construction process
          to create different representations.

Real-world analogy:
    Ordering a custom pizza. You don't get a constructor with
    20 parameters. Instead: start with dough -> add sauce ->
    add cheese -> add toppings, step by step.

When to use:
    - Object has many optional parameters
    - Construction involves multiple steps
    - You want readable, self-documenting creation code
    - You want to create different representations of the same thing

==============================================================
```

```python
"""


# ------------------------------------------------------------
# STEP 1: The Product — what we're building
# ------------------------------------------------------------

class House:
    def __init__(self):
        self.floors = 1
        self.rooms = 1
        self.wall_material = "brick"
        self.roof_type = "flat"
        self.features: list[str] = []

    def __str__(self):
        return (f"  [HOUSE] House: {self.floors}F, {self.rooms}R, "
                f"{self.wall_material} walls, {self.roof_type} roof\n"
                f"     Features: {self.features or 'none'}")


# ------------------------------------------------------------
# STEP 2: The Builder — step-by-step construction
# ------------------------------------------------------------

class HouseBuilder:
    def __init__(self):
        self._house = House()

    def floors(self, count: int) -> "HouseBuilder":
        self._house.floors = count
        return self  # return self for method chaining!

    def rooms(self, count: int) -> "HouseBuilder":
        self._house.rooms = count
        return self

    def walls(self, material: str) -> "HouseBuilder":
        self._house.wall_material = material
        return self

    def roof(self, roof_type: str) -> "HouseBuilder":
        self._house.roof_type = roof_type
        return self

    def with_garage(self) -> "HouseBuilder":
        self._house.features.append("garage")
```

```python
        return self

    def with_pool(self) -> "HouseBuilder":
        self._house.features.append("pool")
        return self

    def with_garden(self) -> "HouseBuilder":
        self._house.features.append("garden")
        return self

    def with_solar(self) -> "HouseBuilder":
        self._house.features.append("solar panels")
        return self

    def with_security(self) -> "HouseBuilder":
        self._house.features.append("security system")
        return self

    def with_smart_home(self) -> "HouseBuilder":
        self._house.features.append("smart home")
        return self

    def build(self) -> House:
        house = self._house
        self._house = House()  # reset for reuse
        return house


# ------------------------------------------------------------
# STEP 3: Optional Director — predefined configurations
# ------------------------------------------------------------

class HouseDirector:
    """Predefined recipes for common house types."""

    @staticmethod
    def luxury_house(builder: HouseBuilder) -> House:
        return (builder
                .floors(3).rooms(8)
                .walls("marble").roof("mansard")
                .with_garage().with_pool().with_garden()
                .with_solar().with_security().with_smart_home()
                .build())

    @staticmethod
    def starter_home(builder: HouseBuilder) -> House:
        return (builder
```

```
                .floors(1).rooms(3)
                .walls("wood").roof("gable")
                .with_garden()
                .build())


# ------------------------------------------------------------
# STEP 4: See it in action
# ------------------------------------------------------------

if __name__ == "__main__":
    builder = HouseBuilder()

    # Custom build — readable and self-documenting!
    print("=== Custom House (method chaining) ===")
    house = (builder
             .floors(2).rooms(5)
             .walls("wood").roof("gable")
             .with_garage().with_solar()
             .build())
    print(house)

    # Using the Director for predefined types
    print("\n=== Luxury House (via Director) ===")
    luxury = HouseDirector.luxury_house(builder)
    print(luxury)

    print("\n=== Starter Home (via Director) ===")
    starter = HouseDirector.starter_home(builder)
    print(starter)

    # Minimal house — just the defaults
    print("\n=== Minimal House (defaults only) ===")
    minimal = builder.build()
    print(minimal)

    print()
    print("=" * 60)
    print("KEY TAKEAWAYS:")
    print("=" * 60)
    print("1. No telescoping constructor — each step is a named method.")
    print("2. Method chaining (.floors(2).rooms(5)) is readable.")
    print("3. Optional features are explicit: .with_pool(), .with_garage().")
    print("4. Director provides predefined recipes.")
    print("5. Same builder process, different results.")
```

Structure

```
+--------------+  uses    +--------------+  builds  +---------+
|   Director   | --------> | HouseBuilder | --------> |  House  |
| (optional)   |          |              |          |(Product)|
| luxury()     |          | floors()     |          +---------+
| starter()    |          | rooms()      |
+--------------+          | with_pool()  |
                          | build()      |
                          +--------------+
```

When to Use

- Object has many optional parameters
- Construction involves multiple steps
- You want readable creation code
- You want predefined configurations (Director)

Key Takeaways

1. No telescoping constructor
2. Method chaining is readable and self-documenting
3. Optional features are explicit named methods
4. Director provides predefined recipes
5. Same builder, different results

Chapter 14: Singleton Pattern

Category: Creational

Intent

Ensure a class has only one instance and provide a global access point.

Real-World Analogy

A country has one president. Everyone asking "who is the president?" gets the same answer.

The Problem

A database connection and app configuration. Multiple instances waste resources. Config changes in one instance don't reflect in others.

```
"""
==============================================================
```

```
 BAD EXAMPLE: Without the Singleton Pattern
================================================================

A database connection and app configuration.

Problems:
    1. Multiple instances waste resources (multiple DB connections)
    2. Config changes in one instance don't reflect in others
    3. No guarantee of a single source of truth
================================================================
"""


class BadDatabase:
    def __init__(self):
        print("  [CONN] Opening NEW database connection...")
        self.connection_id = id(self)

    def query(self, sql: str):
        print(f"  [Conn #{self.connection_id}] Executing: {sql}")


if __name__ == "__main__":
    db1 = BadDatabase()
    db2 = BadDatabase()
    db3 = BadDatabase()

    print(f"\n  db1 is db2? {db1 is db2}")  # False!
    print(f"  Three separate connections created!")

    db1.query("SELECT * FROM users")
    db2.query("SELECT * FROM users")  # wasteful duplicate connection

    print()
    print("3 instances = 3 database connections. Wasteful!")
    print("In production, this can exhaust connection pools.")
    print("-> The Singleton pattern fixes this.")
```

The Solution

Three Python approaches to ensure only one instance exists.

```
"""
================================================================
 DESIGN PATTERN #14: SINGLETON
================================================================
```

```
Category: Creational
Intent:   Ensure a class has only ONE instance, and provide a
          global point of access to it.

Real-world analogy:
    A country has one president at a time. No matter who asks
    "who is the president?", they all get the same person.

When to use:
    - You need exactly one instance (DB connection, config, logger)
    - You need a global access point to that instance
    - The instance should be created lazily (on first use)

CAUTION:
    Singleton is controversial! It's essentially a global variable.
    Overuse leads to tight coupling and hard-to-test code.
    Prefer dependency injection when possible.

==============================================================
"""


# ----------------------------------------------------------
# METHOD 1: Classic Singleton using __new__
# ----------------------------------------------------------

class Database:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance._initialized = False
        return cls._instance

    def __init__(self):
        if self._initialized:
            return
        self._initialized = True
        print("  [CONN] Opening database connection (only once!)")
        self.connection_id = id(self)

    def query(self, sql: str) -> None:
        print(f"  [Conn #{self.connection_id}] Executing: {sql}")


# ----------------------------------------------------------
```

```python
# METHOD 2: Singleton using a metaclass (more Pythonic)
# ------------------------------------------------------

class SingletonMeta(type):
    """A metaclass that creates Singleton instances."""
    _instances = {}

    def __call__(cls, *args, **kwargs):
        if cls not in cls._instances:
            cls._instances[cls] = super().__call__(*args, **kwargs)
        return cls._instances[cls]


class AppConfig(metaclass=SingletonMeta):
    def __init__(self):
        print("  [CFG]  Loading app configuration (only once!)")
        self.settings = {"theme": "dark", "language": "en"}

    def get(self, key: str) -> str:
        return self.settings.get(key, "unknown")

    def set(self, key: str, value: str) -> None:
        self.settings[key] = value
        print(f"  [CFG]  Config updated: {key} = {value}")


# ------------------------------------------------------
# METHOD 3: The simplest way — just use a module!
# ------------------------------------------------------
# In Python, modules are singletons by nature.
# You can just create a module-level instance:
#
#   # config.py
#   settings = {"theme": "dark"}
#
#   # other files
#   from config import settings
#
# This is often the most Pythonic approach.


# ------------------------------------------------------
# See it in action
# ------------------------------------------------------

if __name__ == "__main__":
    print("=== Database Singleton (__new__) ===")
```

```
    db1 = Database()
    db2 = Database()
    db3 = Database()

    print(f"  db1 is db2? {db1 is db2}")  # True!
    print(f"  db1 is db3? {db1 is db3}")  # True!
    db1.query("SELECT * FROM users")
    db2.query("INSERT INTO logs VALUES (...)")

    print("\n=== AppConfig Singleton (metaclass) ===")
    config1 = AppConfig()
    config2 = AppConfig()

    print(f"  config1 is config2? {config1 is config2}")  # True!
    config1.set("theme", "light")
    print(f"  config2 theme: {config2.get('theme')}")  # "light" — same instance!

    print()
    print("=" * 60)
    print("KEY TAKEAWAYS:")
    print("=" * 60)
    print("1. Only ONE instance is ever created.")
    print("2. All callers get the SAME instance.")
    print("3. Python offers 3 ways: __new__, metaclass, or module.")
    print("4. Module-level instance is often the most Pythonic.")
    print("5. CAUTION: Singletons are global state — use sparingly!")
    print("   Prefer dependency injection for testability.")
```

Three Ways in Python

```
# 1. __new__ override
class DB:
    _instance = None
    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

# 2. Metaclass
class SingletonMeta(type):
    _instances = {}
    def __call__(cls, *args, **kwargs):
        if cls not in cls._instances:
            cls._instances[cls] = super().__call__(*args, **kwargs)
        return cls._instances[cls]
```

```
# 3. Module-level instance (most Pythonic)
# config.py -> settings = {"theme": "dark"}
# other.py  -> from config import settings
```

When to Use

- Exactly one instance needed (DB, config, logger)
- Global access point required
- Lazy initialization desired

Caution

Singleton = global state. Overuse leads to tight coupling. Prefer dependency injection when possible.

Key Takeaways

1. Only one instance ever created
2. All callers get the same instance
3. Module-level instance is the most Pythonic way
4. Use sparingly — prefer dependency injection

Chapter 15: Prototype Pattern

Category: Creational

Intent

Create new objects by cloning existing ones, avoiding expensive creation.

Real-World Analogy

Cell division. Instead of building from raw materials, a cell copies itself and the copy can be modified.

The Problem

Creating game enemies. Each enemy type has complex setup. Creating each enemy requires repeating all setup code.

```
"""
============================================================
 BAD EXAMPLE: Without the Prototype Pattern
============================================================
```

```
Creating game enemies. Each enemy type has complex setup.

Problems:
    1. Creating each enemy requires repeating all setup code
    2. If setup is expensive (loading assets), it's wasteful
    3. No easy way to create variations of existing objects
    4. Copy logic scattered and error-prone
==============================================================
"""


class BadEnemy:
    def __init__(self, name, hp, attack, defense, speed, abilities, sprite):
        self.name = name
        self.hp = hp
        self.attack = attack
        self.defense = defense
        self.speed = speed
        self.abilities = abilities
        self.sprite = sprite  # imagine this loads a big file

    def __str__(self):
        return f"  {self.name}: HP={self.hp}, ATK={self.attack}, DEF={self.defense}"


if __name__ == "__main__":
    # Creating 3 goblins — repeating ALL parameters every time!
    goblin1 = BadEnemy("Goblin", 50, 10, 5, 7, ["scratch", "bite"], "goblin.png")
    goblin2 = BadEnemy("Goblin", 50, 10, 5, 7, ["scratch", "bite"], "goblin.png")
    goblin3 = BadEnemy("Goblin", 50, 10, 5, 7, ["scratch", "bite"], "goblin.png")

    # Want a slightly stronger goblin? Repeat everything + change one field
    goblin_chief = BadEnemy("Goblin Chief", 80, 15, 8, 7, ["scratch", "bite", "rally"],

    for g in [goblin1, goblin2, goblin3, goblin_chief]:
        print(g)

    print()
    print("Repeated all 7 parameters for every goblin.")
    print("-> The Prototype pattern fixes this.")
```

The Solution

Clone prototypes and apply overrides for variations.

```
"""
==============================================================
```

```
 DESIGN PATTERN #15: PROTOTYPE
================================================================

Category: Creational
Intent:   Create new objects by cloning existing ones, avoiding
          the cost of creation from scratch.

Real-world analogy:
    Cell division. Instead of building a new cell from raw
    materials, an existing cell copies itself and then the
    copy can be modified.

When to use:
    - Object creation is expensive (loading files, DB queries)
    - You need many similar objects with small variations
    - You want to avoid subclassing just to change defaults

================================================================
"""

import copy


# ------------------------------------------------------------
# STEP 1: The Prototype — supports cloning
# ------------------------------------------------------------

class Enemy:
    def __init__(self, name: str, hp: int, attack: int,
                 defense: int, speed: int, abilities: list[str],
                 sprite: str):
        self.name = name
        self.hp = hp
        self.attack = attack
        self.defense = defense
        self.speed = speed
        self.abilities = abilities
        self.sprite = sprite
        # Imagine expensive initialization here:
        # self._load_sprite(sprite)
        # self._calculate_stats()

    def clone(self) -> "Enemy":
        """Deep copy — the clone is fully independent."""
        return copy.deepcopy(self)

    def __str__(self):
```

```
        return (f"  {self.name}: HP={self.hp}, ATK={self.attack}, "
                f"DEF={self.defense}, SPD={self.speed}, "
                f"Abilities={self.abilities}")


# ---------------------------------------------------------
# STEP 2: A registry of prototypes (optional but useful)
# ---------------------------------------------------------

class EnemyRegistry:
    def __init__(self):
        self._prototypes: dict[str, Enemy] = {}

    def register(self, key: str, prototype: Enemy) -> None:
        self._prototypes[key] = prototype

    def create(self, key: str, **overrides) -> Enemy:
        """Clone a prototype and apply overrides."""
        if key not in self._prototypes:
            raise KeyError(f"No prototype registered for '{key}'")
        clone = self._prototypes[key].clone()
        for attr, value in overrides.items():
            setattr(clone, attr, value)
        return clone


# ---------------------------------------------------------
# STEP 3: See it in action
# ---------------------------------------------------------

if __name__ == "__main__":
    # Create prototypes (expensive, done once)
    registry = EnemyRegistry()

    registry.register("goblin", Enemy(
        name="Goblin", hp=50, attack=10, defense=5,
        speed=7, abilities=["scratch", "bite"], sprite="goblin.png"
    ))

    registry.register("dragon", Enemy(
        name="Dragon", hp=500, attack=80, defense=50,
        speed=30, abilities=["fire_breath", "tail_swipe", "fly"],
        sprite="dragon.png"
    ))

    # Spawn enemies by cloning — fast and clean!
    print("=== Spawn Goblins (cloned from prototype) ===")
```

```python
    for i in range(3):
        goblin = registry.create("goblin")
        print(goblin)

    print("\n=== Spawn Goblin Chief (clone + override) ===")
    chief = registry.create("goblin",
                            name="Goblin Chief", hp=80, attack=15,
                            abilities=["scratch", "bite", "rally"])
    print(chief)

    print("\n=== Spawn Dragons ===")
    dragon = registry.create("dragon")
    print(dragon)

    baby_dragon = registry.create("dragon",
                                  name="Baby Dragon", hp=100, attack=20)
    print(baby_dragon)

    # Verify clones are independent
    print("\n=== Independence check ===")
    g1 = registry.create("goblin")
    g2 = registry.create("goblin")
    g1.name = "Modified Goblin"
    g1.abilities.append("steal")
    print(f"  g1: {g1.name}, abilities={g1.abilities}")
    print(f"  g2: {g2.name}, abilities={g2.abilities}")  # unaffected!

    print()
    print("=" * 60)
    print("KEY TAKEAWAYS:")
    print("=" * 60)
    print("1. Clone existing objects instead of creating from scratch.")
    print("2. Deep copy ensures clones are fully independent.")
    print("3. Registry provides a catalog of reusable prototypes.")
    print("4. Overrides allow easy variations (Goblin -> Goblin Chief).")
    print("5. Python's copy.deepcopy() does the heavy lifting.")
```

Structure

```
+---------------+  clone()  +--------------+
|   Prototype   | ---------> |    Clone     |
|   (Goblin)    |            |  (new Goblin)|
| hp=50, atk=10 |            | hp=50, atk=10|
+---------------+            +--------------+

+-----------------+
|  EnemyRegistry  |  register("goblin", prototype)
```

```
|                  |  create("goblin", hp=80)  -> clone + override
+-----------------+
```

When to Use

- Object creation is expensive
- Many similar objects with small variations needed
- Avoid subclassing just for different defaults

Key Takeaways

1. Clone instead of create from scratch
2. Deep copy ensures independence
3. Registry catalogs reusable prototypes
4. Overrides allow easy variations
5. Python's `copy.deepcopy()` does the heavy lifting

Part III: Structural Patterns

Chapter 16: Decorator Pattern

Category: Structural

Intent

Attach additional responsibilities to an object dynamically. A flexible alternative to subclassing.

Real-World Analogy

Wearing clothes. You add a shirt, jacket, scarf — each layer adds warmth without changing you. Add/remove layers freely.

The Problem

A coffee shop with different toppings/extras. Without the pattern, you need 2^N classes for N toppings.

```
"""
============================================================
 BAD EXAMPLE: Without the Decorator Pattern
============================================================

A coffee shop with different toppings/extras.

Problems:
    1. Class explosion: Coffee, CoffeeWithMilk, CoffeeWithMilkAndSugar,
       CoffeeWithMilkAndSugarAndWhip... 2^N combinations!
    2. Can't add toppings dynamically at runtime
    3. Every new topping doubles the number of classes
============================================================
"""
```

```python
class Coffee:
    def cost(self): return 2.00
    def description(self): return "Coffee"

class CoffeeWithMilk:
    def cost(self): return 2.50
    def description(self): return "Coffee + Milk"

class CoffeeWithMilkAndSugar:
    def cost(self): return 2.70
    def description(self): return "Coffee + Milk + Sugar"

class CoffeeWithMilkAndSugarAndWhip:
    def cost(self): return 3.40
    def description(self): return "Coffee + Milk + Sugar + Whipped Cream"

# CoffeeWithSugar? CoffeeWithWhip? CoffeeWithMilkAndWhip?
# 4 toppings = 16 possible classes!!


if __name__ == "__main__":
    orders = [Coffee(), CoffeeWithMilk(), CoffeeWithMilkAndSugar()]
    for o in orders:
        print(f"  {o.description()}: ${o.cost():.2f}")

    print()
    print("4 toppings = 2^4 = 16 classes needed!")
    print("-> The Decorator pattern fixes this.")
```

The Solution

Wrap objects in decorator layers. Each decorator adds behavior and delegates to the wrapped object.

```python
"""
============================================================
 DESIGN PATTERN #16: DECORATOR
============================================================

Category: Structural
Intent:   Attach additional responsibilities to an object dynamically.
          Decorators provide a flexible alternative to subclassing
          for extending functionality.

Real-world analogy:
    Wearing clothes. You (the base object) can add a shirt,
    then a jacket, then a scarf. Each layer adds functionality
```

```
    (warmth) without changing YOU. You can add/remove layers
    freely.

When to use:
    - You need to add behavior to objects dynamically
    - Subclassing would cause a class explosion
    - You want to combine behaviors freely (mix and match)

============================================================
"""

from abc import ABC, abstractmethod


# ------------------------------------------------------------
# STEP 1: Component interface
# ------------------------------------------------------------

class Beverage(ABC):
    @abstractmethod
    def cost(self) -> float:
        pass

    @abstractmethod
    def description(self) -> str:
        pass


# ------------------------------------------------------------
# STEP 2: Concrete components (base objects)
# ------------------------------------------------------------

class Coffee(Beverage):
    def cost(self) -> float:
        return 2.00

    def description(self) -> str:
        return "Coffee"


class Tea(Beverage):
    def cost(self) -> float:
        return 1.50

    def description(self) -> str:
        return "Tea"
```

```python
# ------------------------------------------------------------
# STEP 3: Decorator base class
# ------------------------------------------------------------
# A decorator IS a Beverage AND HAS a Beverage (wraps it).

class BeverageDecorator(Beverage, ABC):
    def __init__(self, beverage: Beverage):
        self._beverage = beverage


# ------------------------------------------------------------
# STEP 4: Concrete decorators (toppings)
# ------------------------------------------------------------

class Milk(BeverageDecorator):
    def cost(self) -> float:
        return self._beverage.cost() + 0.50

    def description(self) -> str:
        return self._beverage.description() + " + Milk"


class Sugar(BeverageDecorator):
    def cost(self) -> float:
        return self._beverage.cost() + 0.20

    def description(self) -> str:
        return self._beverage.description() + " + Sugar"


class WhippedCream(BeverageDecorator):
    def cost(self) -> float:
        return self._beverage.cost() + 0.70

    def description(self) -> str:
        return self._beverage.description() + " + Whipped Cream"


class Caramel(BeverageDecorator):
    def cost(self) -> float:
        return self._beverage.cost() + 0.60

    def description(self) -> str:
        return self._beverage.description() + " + Caramel"


# ------------------------------------------------------------
```

```python
# STEP 5: See it in action
# ----------------------------------------------------------

if __name__ == "__main__":
    # Build orders by wrapping decorators
    print("=== Orders ===")

    # Simple coffee
    order1 = Coffee()
    print(f"  {order1.description()}: ${order1.cost():.2f}")

    # Coffee with milk and sugar
    order2 = Sugar(Milk(Coffee()))
    print(f"  {order2.description()}: ${order2.cost():.2f}")

    # Fancy coffee: milk + sugar + whipped cream + caramel
    order3 = Caramel(WhippedCream(Sugar(Milk(Coffee()))))
    print(f"  {order3.description()}: ${order3.cost():.2f}")

    # Tea with milk
    order4 = Milk(Tea())
    print(f"  {order4.description()}: ${order4.cost():.2f}")

    # Double milk! (can apply same decorator twice)
    order5 = Milk(Milk(Coffee()))
    print(f"  {order5.description()}: ${order5.cost():.2f}")

    print(f"\n  Total classes: 2 bases + 4 decorators = 6")
    print(f"  Possible combinations: unlimited!")

    print()
    print("=" * 60)
    print("KEY TAKEAWAYS:")
    print("=" * 60)
    print("1. 4 toppings + 2 bases = 6 classes (not 32!).")
    print("2. Decorators wrap objects like layers of an onion.")
    print("3. Can apply the same decorator multiple times (double milk).")
    print("4. Add/remove behaviors dynamically at runtime.")
    print("5. Python also has @decorator syntax — same core idea!")
```

Structure

```
       Caramel( WhippedCream( Sugar( Milk( Coffee() ) ) ) )
          |          |          |       |       |
          +--wraps---+--wraps---+-wraps-+-wraps-+

  Each decorator IS a Beverage AND wraps a Beverage.
```

```
cost() = own cost + wrapped.cost()
```

The Math

- Without pattern: N toppings = 2^N classes
- With pattern: N toppings = N + base classes

When to Use

- Add behavior dynamically at runtime
- Subclassing causes class explosion
- Mix and match behaviors freely

Key Takeaways

1. Decorators wrap objects like layers of an onion
2. 4 toppings = 6 classes, not 32
3. Same decorator can be applied multiple times
4. Python's `@decorator` syntax is the same concept
5. Each decorator has the same interface as the wrapped object

---

Chapter 17: Adapter Pattern

Category: Structural

Intent

Convert the interface of a class into another interface clients expect.

Real-World Analogy

A power adapter for travel. Your US plug doesn't fit a European outlet. The adapter converts one to the other without modifying either.

The Problem

Your app uses a PaymentProcessor interface, but third-party libraries have completely different interfaces. You can't modify them.

```
"""
============================================================
 BAD EXAMPLE: Without the Adapter Pattern
============================================================

Your app uses a PaymentProcessor interface, but a third-party
library (StripeAPI) has a completely different interface.
```

```
Problems:
    1. Can't use StripeAPI directly — incompatible method names
    2. Modifying StripeAPI is impossible (it's a third-party lib)
    3. Client code would need ugly if/elif for each payment service
==============================================================
"""


class PaymentProcessor:
    """Your app's expected interface."""
    def pay(self, amount: float) -> str:
        raise NotImplementedError


class StripeAPI:
    """Third-party library — you can't modify this!"""
    def create_charge(self, amount_cents: int, currency: str) -> dict:
        return {"status": "success", "amount": amount_cents, "currency": currency}


class PayPalAPI:
    """Another third-party library with yet another interface."""
    def send_payment(self, recipient: str, value: float) -> bool:
        return True


if __name__ == "__main__":
    stripe = StripeAPI()
    # Can't do stripe.pay(10.00) — method doesn't exist!
    # Must use stripe.create_charge(1000, "usd") instead.

    result = stripe.create_charge(1000, "usd")
    print(f"  Stripe raw call: {result}")

    print()
    print("StripeAPI.create_charge() != PaymentProcessor.pay()")
    print("PayPalAPI.send_payment() != PaymentProcessor.pay()")
    print("Incompatible interfaces!")
    print("-> The Adapter pattern fixes this.")
```

The Solution

Create adapter classes that translate between your interface and the third-party interface.

```
"""
==============================================================
```

```
 DESIGN PATTERN #17: ADAPTER
============================================================

Category: Structural
Intent:   Convert the interface of a class into another interface
          that clients expect. Lets classes work together that
          couldn't otherwise because of incompatible interfaces.

Real-world analogy:
    A power adapter for international travel. Your laptop has a
    US plug, but the outlet is European. The adapter converts
    one interface to another without modifying either.

When to use:
    - You want to use a class with an incompatible interface
    - You can't modify the existing class (third-party library)
    - You want to create a reusable bridge between interfaces

============================================================
"""

from abc import ABC, abstractmethod


# ------------------------------------------------------------
# STEP 1: The target interface (what YOUR code expects)
# ------------------------------------------------------------

class PaymentProcessor(ABC):
    @abstractmethod
    def pay(self, amount: float) -> str:
        pass


# ------------------------------------------------------------
# STEP 2: Third-party services (can't modify these!)
# ------------------------------------------------------------

class StripeAPI:
    def create_charge(self, amount_cents: int, currency: str) -> dict:
        return {"status": "success", "amount": amount_cents, "currency": currency}


class PayPalAPI:
    def send_payment(self, recipient: str, value: float) -> bool:
        return True
```

```python
class SquareAPI:
    def process_transaction(self, amount: str) -> str:
        return f"APPROVED:{amount}"


# ------------------------------------------------------------
# STEP 3: Adapters — bridge incompatible interfaces
# ------------------------------------------------------------

class StripeAdapter(PaymentProcessor):
    def __init__(self, stripe: StripeAPI):
        self._stripe = stripe

    def pay(self, amount: float) -> str:
        # Convert dollars to cents, add currency
        result = self._stripe.create_charge(int(amount * 100), "usd")
        return f"[CARD] Stripe: {result['status']} ($${amount:.2f})"


class PayPalAdapter(PaymentProcessor):
    def __init__(self, paypal: PayPalAPI):
        self._paypal = paypal

    def pay(self, amount: float) -> str:
        # Adapt to PayPal's different method signature
        success = self._paypal.send_payment("merchant@shop.com", amount)
        status = "success" if success else "failed"
        return f"[PP]  PayPal: {status} ($${amount:.2f})"


class SquareAdapter(PaymentProcessor):
    def __init__(self, square: SquareAPI):
        self._square = square

    def pay(self, amount: float) -> str:
        # Adapt to Square's string-based interface
        result = self._square.process_transaction(f"{amount:.2f}")
        return f"[SQ] Square: {result}"


# ------------------------------------------------------------
# STEP 4: Client code — works with any PaymentProcessor
# ------------------------------------------------------------

def checkout(processor: PaymentProcessor, amount: float) -> None:
    """This function has NO idea which payment service is used."""
    result = processor.pay(amount)
```

```python
    print(f"  {result}")


# ------------------------------------------------------------
# STEP 5: See it in action
# ------------------------------------------------------------

if __name__ == "__main__":
    # Wrap third-party APIs in adapters
    stripe = StripeAdapter(StripeAPI())
    paypal = PayPalAdapter(PayPalAPI())
    square = SquareAdapter(SquareAPI())

    # Client code uses the SAME interface for all!
    print("=== Processing payments ===")
    checkout(stripe, 29.99)
    checkout(paypal, 49.99)
    checkout(square, 19.99)

    # Can even loop through different processors
    print("\n=== Batch payment ===")
    processors = [stripe, paypal, square]
    for processor in processors:
        checkout(processor, 10.00)

    print()
    print("=" * 60)
    print("KEY TAKEAWAYS:")
    print("=" * 60)
    print("1. Third-party APIs were NOT modified.")
    print("2. Each adapter translates one interface to another.")
    print("3. Client code (checkout) works with ANY payment service.")
    print("4. Adding a new service = just write a new adapter.")
    print("5. The adapter handles all conversion (dollars->cents, etc.).")
```

Structure

```
+----------+  calls   +----------------+  translates  +-----------+
|  Client  | --------> | StripeAdapter  | ------------> | StripeAPI |
|          |  pay()   | (Adapter)      |  create_     | (Adaptee) |
|checkout()|          |  pay() ->       |  charge()    |           |
+----------+          +----------------+              +-----------+
```

When to Use

- Incompatible interface from a third-party library
- Can't modify the existing class

- Want a reusable bridge between interfaces

Key Takeaways

1. Third-party APIs are NOT modified
2. Each adapter translates between interfaces
3. Client works uniformly with any adapted service
4. Handles all conversion logic (units, formats, etc.)

Chapter 18: Facade Pattern

Category: Structural

Intent

Provide a simplified interface to a complex subsystem.

Real-World Analogy

A hotel concierge. Tell them "plan my evening" and they handle restaurant, taxi, and theater bookings behind the scenes.

The Problem

Starting a home theater system requires many complex steps across multiple subsystems.

```
"""
============================================================
 BAD EXAMPLE: Without the Facade Pattern
============================================================

Starting a home theater system requires many complex steps
across multiple subsystems.

Problems:
    1. Client must know about ALL subsystems and their APIs
    2. Complex sequence of calls that must be in the right order
    3. Every client repeats the same boilerplate
    4. Change one subsystem -> update every client
============================================================
"""


class TV:
    def on(self): print("  TV: turning on")
```

```
    def set_input(self, src): print(f"  TV: input set to {src}")

class Amplifier:
    def on(self): print("  Amplifier: turning on")
    def set_volume(self, vol): print(f"  Amplifier: volume set to {vol}")
    def set_source(self, src): print(f"  Amplifier: source set to {src}")

class StreamingPlayer:
    def on(self): print("  StreamingPlayer: turning on")
    def play(self, movie): print(f"  StreamingPlayer: playing '{movie}'")

class Lights:
    def dim(self, level): print(f"  Lights: dimmed to {level}%")


if __name__ == "__main__":
    # Client must orchestrate ALL of this manually!
    tv = TV()
    amp = Amplifier()
    player = StreamingPlayer()
    lights = Lights()

    print("=== Watch movie (manual steps) ===")
    lights.dim(20)
    tv.on()
    tv.set_input("HDMI1")
    amp.on()
    amp.set_source("HDMI1")
    amp.set_volume(25)
    player.on()
    player.play("The Matrix")

    print()
    print("7 steps across 4 objects, in the right order!")
    print("Every client that wants to watch a movie repeats this.")
    print("-> The Facade pattern fixes this.")
```

The Solution

Create a facade that provides simple methods wrapping the complex subsystem interactions.

```
"""
================================================================
 DESIGN PATTERN #18: FACADE
================================================================
```

```
Category: Structural
Intent:   Provide a simplified interface to a complex subsystem.

Real-world analogy:
    A hotel concierge. Instead of calling the restaurant, taxi,
    and theater yourself, you tell the concierge "plan my evening"
    and they handle everything behind the scenes.

When to use:
    - A subsystem is complex with many interdependent classes
    - You want a simple interface for common use cases
    - You want to decouple clients from subsystem details

==============================================================
"""



# ---------------------------------------------------------
# STEP 1: Complex subsystem classes (unchanged)
# ---------------------------------------------------------

class TV:
    def on(self):
        print("  [TV] TV: turning on")

    def off(self):
        print("  [TV] TV: turning off")

    def set_input(self, source: str):
        print(f"  [TV] TV: input -> {source}")


class Amplifier:
    def on(self):
        print("  [AMP] Amplifier: turning on")

    def off(self):
        print("  [AMP] Amplifier: turning off")

    def set_volume(self, level: int):
        print(f"  [AMP] Amplifier: volume -> {level}")

    def set_source(self, source: str):
        print(f"  [AMP] Amplifier: source -> {source}")


class StreamingPlayer:
```

```python
    def on(self):
        print("  [PLAY] Player: turning on")

    def off(self):
        print("  [PLAY] Player: turning off")

    def play(self, movie: str):
        print(f"  [PLAY] Player: playing '{movie}'")

    def stop(self):
        print("  [PLAY] Player: stopped")


class Lights:
    def dim(self, level: int):
        print(f"  [LIGHT] Lights: dimmed to {level}%")

    def on(self):
        print("  [LIGHT] Lights: turned on (100%)")


# ------------------------------------------------------------
# STEP 2: The Facade — simple interface to complex system
# ------------------------------------------------------------

class HomeTheaterFacade:
    def __init__(self):
        self._tv = TV()
        self._amp = Amplifier()
        self._player = StreamingPlayer()
        self._lights = Lights()

    def watch_movie(self, movie: str) -> None:
        """One method replaces 7+ manual steps."""
        print(f"  [MOVIE] Getting ready to watch '{movie}'...")
        self._lights.dim(20)
        self._tv.on()
        self._tv.set_input("HDMI1")
        self._amp.on()
        self._amp.set_source("HDMI1")
        self._amp.set_volume(25)
        self._player.on()
        self._player.play(movie)
        print(f"  [ENJOY] Enjoy the movie!")

    def end_movie(self) -> None:
        """Shutdown is also simplified."""
```

```
        print("  [MOVIE] Shutting down movie theater...")
        self._player.stop()
        self._player.off()
        self._amp.off()
        self._tv.off()
        self._lights.on()
        print("  [BYE] Good night!")

    def listen_music(self) -> None:
        """Another simplified operation."""
        print("  [MUSIC] Setting up for music...")
        self._lights.dim(50)
        self._amp.on()
        self._amp.set_source("Bluetooth")
        self._amp.set_volume(15)
        print("  [MUSIC] Ready for music!")


# ------------------------------------------------------------
# STEP 3: See it in action
# ------------------------------------------------------------

if __name__ == "__main__":
    # Client only needs ONE object and simple method calls
    theater = HomeTheaterFacade()

    print("=== Watch Movie ===")
    theater.watch_movie("The Matrix")

    print("\n=== End Movie ===")
    theater.end_movie()

    print("\n=== Listen to Music ===")
    theater.listen_music()

    print()
    print("=" * 60)
    print("KEY TAKEAWAYS:")
    print("=" * 60)
    print("1. Client calls ONE method instead of 7+ steps.")
    print("2. Subsystem classes are unchanged and still accessible.")
    print("3. Facade doesn't add new functionality — it simplifies.")
    print("4. Multiple facades can exist for different use cases.")
    print("5. The subsystem is hidden but NOT locked away.")
```

Structure

```
+------------+  simple call  +-----------------------------+
|   Client   | -------------> |   HomeTheaterFacade         |
|            | watch_movie() |                             |
+------------+               |  TV + Amp + Player + Lights |
                             +-----------------------------+
                                  |    |    |    |
                              manages all subsystems
```

Facade vs Adapter

	Adapter	Facade
Purpose	Make incompatible interface compatible	Simplify a complex interface
Scope	Wraps ONE class	Wraps MANY classes
Changes interface?	Yes (translates)	Yes (simplifies)

Key Takeaways

1. One method replaces many complex steps
2. Subsystem unchanged and still directly accessible
3. Facade simplifies, doesn't add new functionality
4. Multiple facades can coexist

Chapter 19: Composite Pattern

Category: Structural

Intent

Compose objects into tree structures. Let clients treat individual objects and compositions uniformly.

Real-World Analogy

An army. "Attack" works the same for a soldier or an entire division — the command cascades down the hierarchy.

The Problem

A file system with files and folders. Client must check type (file vs folder) everywhere. Nested folders require recursive logic in the client.

```python
"""
================================================================
 BAD EXAMPLE: Without the Composite Pattern
================================================================

A file system with files and folders. You want to calculate
the total size.

Problems:
    1. Client must check type (file vs folder) everywhere
    2. Nested folders require recursive logic in the client
    3. Files and folders have different interfaces
    4. Adding new types (symlinks) breaks all client code
================================================================
"""


class BadFile:
    def __init__(self, name: str, size: int):
        self.name = name
        self.size = size


class BadFolder:
    def __init__(self, name: str):
        self.name = name
        self.children = []

    def add(self, item):
        self.children.append(item)


def calculate_size(item) -> int:
    """Client must know about EVERY type and handle recursion."""
    if isinstance(item, BadFile):
        return item.size
    elif isinstance(item, BadFolder):
        total = 0
        for child in item.children:
            total += calculate_size(child)  # manual recursion
        return total
    else:
        raise TypeError(f"Unknown type: {type(item)}")


if __name__ == "__main__":
    root = BadFolder("root")
```

```
    root.add(BadFile("readme.txt", 100))
    docs = BadFolder("docs")
    docs.add(BadFile("guide.pdf", 500))
    docs.add(BadFile("api.md", 200))
    root.add(docs)

    print(f"  Total size: {calculate_size(root)} bytes")
    print()
    print("Client must use isinstance() and handle recursion itself.")
    print("-> The Composite pattern fixes this.")
```

The Solution

Give files and folders the same interface. Operations cascade down the tree automatically.

```
"""
==============================================================
 DESIGN PATTERN #19: COMPOSITE
==============================================================

Category: Structural
Intent:   Compose objects into tree structures to represent
          part-whole hierarchies. Let clients treat individual
          objects and compositions uniformly.

Real-world analogy:
    An army. A general commands divisions, which contain brigades,
    which contain platoons, which contain soldiers. The order
    "attack" works the same whether given to a single soldier
    or an entire division — it cascades down the tree.

When to use:
    - You have a tree/hierarchy structure
    - You want to treat leaves and branches the same way
    - Operations should cascade down the tree

==============================================================
"""

from abc import ABC, abstractmethod


# ----------------------------------------------------------
# STEP 1: Component interface — same for leaves and branches
# ----------------------------------------------------------
```

```python
class FileSystemItem(ABC):
    def __init__(self, name: str):
        self.name = name

    @abstractmethod
    def get_size(self) -> int:
        pass

    @abstractmethod
    def display(self, indent: int = 0) -> None:
        pass


# ---------------------------------------------------------
# STEP 2: Leaf — individual objects (files)
# ---------------------------------------------------------

class File(FileSystemItem):
    def __init__(self, name: str, size: int):
        super().__init__(name)
        self.size = size

    def get_size(self) -> int:
        return self.size

    def display(self, indent: int = 0) -> None:
        print(f"{'  ' * indent}[FILE] {self.name} ({self.size}B)")


# ---------------------------------------------------------
# STEP 3: Composite — contains other items (folders)
# ---------------------------------------------------------

class Folder(FileSystemItem):
    def __init__(self, name: str):
        super().__init__(name)
        self._children: list[FileSystemItem] = []

    def add(self, item: FileSystemItem) -> None:
        self._children.append(item)

    def remove(self, item: FileSystemItem) -> None:
        self._children.remove(item)

    def get_size(self) -> int:
        """Size cascades down the tree automatically."""
        return sum(child.get_size() for child in self._children)
```

```python
    def display(self, indent: int = 0) -> None:
        print(f"{'  ' * indent}[DIR] {self.name}/ ({self.get_size()}B)")
        for child in self._children:
            child.display(indent + 1)


# ------------------------------------------------------------
# STEP 4: See it in action
# ------------------------------------------------------------

if __name__ == "__main__":
    # Build a file tree
    root = Folder("project")

    src = Folder("src")
    src.add(File("main.py", 1200))
    src.add(File("utils.py", 800))

    models = Folder("models")
    models.add(File("user.py", 500))
    models.add(File("product.py", 600))
    src.add(models)

    tests = Folder("tests")
    tests.add(File("test_main.py", 900))
    tests.add(File("test_utils.py", 700))

    root.add(src)
    root.add(tests)
    root.add(File("README.md", 300))
    root.add(File(".gitignore", 50))

    # Display the tree
    print("=== File System Tree ===")
    root.display()

    # get_size() works the same on files AND folders
    print(f"\n=== Sizes ===")
    print(f"  Total project size: {root.get_size()}B")
    print(f"  src/ size: {src.get_size()}B")
    print(f"  models/ size: {models.get_size()}B")
    print(f"  Single file (main.py): {src._children[0].get_size()}B")

    print()
    print("=" * 60)
    print("KEY TAKEAWAYS:")
    print("=" * 60)
```

```
    print("1. File and Folder share the SAME interface.")
    print("2. get_size() works identically on both — no isinstance().")
    print("3. Tree structure with automatic cascading.")
    print("4. Client treats leaves and branches uniformly.")
    print("5. Perfect for: file systems, UI components, org charts.")
```

Structure

```
        FileSystemItem (interface)
           get_size()
           display()
          /          \
     File (leaf)    Folder (composite)
     get_size()     get_size() -> sum of children
                    add() / remove()
                    children: [FileSystemItem...]
```

When to Use

- Tree/hierarchy structures
- Treat leaves and branches the same
- Operations cascade down the tree

Key Takeaways

1. Leaf and Composite share the same interface
2. No isinstance() checks needed
3. Operations cascade automatically
4. Perfect for: file systems, UI trees, org charts, menus

Chapter 20: Proxy Pattern

Category: Structural

Intent

Provide a surrogate or placeholder to control access to another object.

Real-World Analogy

A credit card is a proxy for your bank account. Same "pay" interface, but adds security checks, spending limits, and logging.

The Problem

A heavy database that loads ALL data on initialization, with no access control, no caching, and no logging.

```
"""
================================================================
 BAD EXAMPLE: Without the Proxy Pattern
================================================================

A heavy database that loads ALL data on initialization,
and has no access control.

Problems:
    1. Expensive resource loaded even if never used
    2. No access control — anyone can read/write
    3. No caching — same query executed repeatedly
    4. No logging — can't track who accessed what
================================================================
"""


class BadDatabase:
    def __init__(self):
        print("  [CONN] Loading entire database into memory... (5 seconds)")
        self.data = {"users": 1000, "orders": 5000, "products": 300}

    def query(self, table: str) -> int:
        return self.data.get(table, 0)


if __name__ == "__main__":
    # Database loaded immediately, even if we only need one query
    db = BadDatabase()
    print(f"  Users: {db.query('users')}")

    # No access control
    print(f"  Orders: {db.query('orders')}")  # anyone can query anything

    # Same query again — no caching
    print(f"  Users again: {db.query('users')}")  # hits DB again

    print()
    print("Problems: eager loading, no access control, no caching.")
    print("-> The Proxy pattern fixes this.")
```

The Solution

Create proxy objects that control access: lazy loading, caching, and protection.

```
"""
================================================================
 DESIGN PATTERN #20: PROXY
================================================================

Category: Structural
Intent:   Provide a surrogate or placeholder for another object
          to control access to it.

Real-world analogy:
    A credit card is a proxy for your bank account. It provides
    the same "pay" interface but adds security checks, spending
    limits, and transaction logging — all without modifying
    your actual bank account.

Types of proxies:
    - Virtual Proxy: lazy initialization (load when first used)
    - Protection Proxy: access control (check permissions)
    - Caching Proxy: store results to avoid repeated work
    - Logging Proxy: log all interactions

When to use:
    - You want lazy initialization of expensive objects
    - You need access control
    - You want caching, logging, or rate limiting
    - The proxy has the SAME interface as the real object

================================================================
"""

from abc import ABC, abstractmethod


# ------------------------------------------------------------
# STEP 1: Subject interface — same for real and proxy
# ------------------------------------------------------------

class Database(ABC):
    @abstractmethod
    def query(self, table: str) -> int:
        pass


# ------------------------------------------------------------
# STEP 2: Real subject — the expensive object
```

```python
# ------------------------------------------------------------

class RealDatabase(Database):
    def __init__(self):
        print("  [CONN] Loading entire database into memory...")
        self.data = {"users": 1000, "orders": 5000, "products": 300}

    def query(self, table: str) -> int:
        return self.data.get(table, 0)


# ------------------------------------------------------------
# STEP 3: Virtual Proxy — lazy loading
# ------------------------------------------------------------

class LazyDatabaseProxy(Database):
    def __init__(self):
        self._real_db: RealDatabase | None = None

    def query(self, table: str) -> int:
        if self._real_db is None:
            print("  [WAIT] First access — initializing database...")
            self._real_db = RealDatabase()
        return self._real_db.query(table)


# ------------------------------------------------------------
# STEP 4: Protection Proxy — access control
# ------------------------------------------------------------

class ProtectedDatabaseProxy(Database):
    def __init__(self, db: Database, user_role: str):
        self._db = db
        self._user_role = user_role
        self._restricted = {"orders", "products"}

    def query(self, table: str) -> int:
        if table in self._restricted and self._user_role != "admin":
            print(f"  [NO] Access denied: '{self._user_role}' cannot query '{table}'")
            return -1
        return self._db.query(table)


# ------------------------------------------------------------
# STEP 5: Caching Proxy — avoid repeated work
# ------------------------------------------------------------
```

```python
class CachingDatabaseProxy(Database):
    def __init__(self, db: Database):
        self._db = db
        self._cache: dict[str, int] = {}

    def query(self, table: str) -> int:
        if table in self._cache:
            print(f"  [FAST] Cache hit for '{table}'")
            return self._cache[table]
        print(f"  [SEARCH] Cache miss for '{table}' — querying database...")
        result = self._db.query(table)
        self._cache[table] = result
        return result


# ------------------------------------------------------------
# STEP 6: See it in action
# ------------------------------------------------------------

if __name__ == "__main__":
    print("=== Virtual Proxy (lazy loading) ===")
    lazy_db = LazyDatabaseProxy()
    print("  Database NOT loaded yet!")
    print(f"  Users: {lazy_db.query('users')}")  # loads here
    print(f"  Orders: {lazy_db.query('orders')}")  # already loaded

    print("\n=== Caching Proxy ===")
    real_db = RealDatabase()
    cached_db = CachingDatabaseProxy(real_db)
    print(f"  Users: {cached_db.query('users')}")  # miss
    print(f"  Users: {cached_db.query('users')}")  # hit!
    print(f"  Orders: {cached_db.query('orders')}")  # miss
    print(f"  Orders: {cached_db.query('orders')}")  # hit!

    print("\n=== Protection Proxy ===")
    protected_db = ProtectedDatabaseProxy(real_db, "viewer")
    print(f"  Users: {protected_db.query('users')}")  # allowed
    print(f"  Orders: {protected_db.query('orders')}")  # denied!

    protected_admin = ProtectedDatabaseProxy(real_db, "admin")
    print(f"  Orders (admin): {protected_admin.query('orders')}")  # allowed

    print("\n=== Stacking Proxies (Lazy + Cached + Protected) ===")
    stacked = ProtectedDatabaseProxy(
        CachingDatabaseProxy(LazyDatabaseProxy()),
        "admin"
    )
```

```
    print(f"  Users: {stacked.query('users')}")
    print(f"  Users: {stacked.query('users')}")

    print()
    print("=" * 60)
    print("KEY TAKEAWAYS:")
    print("=" * 60)
    print("1. Proxy has the SAME interface as the real object.")
    print("2. Virtual proxy: lazy init (load only when needed).")
    print("3. Protection proxy: access control.")
    print("4. Caching proxy: avoid repeated expensive operations.")
    print("5. Proxies can be stacked like decorators!")
```

Types of Proxies

Type	Purpose	Example
Virtual	Lazy initialization	Load DB only when first queried
Protection	Access control	Check user role before allowing
Caching	Avoid repeated work	Store query results
Logging	Track interactions	Log every access

Structure

```
+--------+  same interface  +-----------+  delegates  +--------------+
| Client | ----------------> |   Proxy   | -----------> | RealDatabase |
|        |  query()         |  query()  |             |   query()    |
+--------+                  | + control |             +--------------+
                            +-----------+
```

Proxy vs Decorator

- **Decorator:** adds new behavior dynamically
- **Proxy:** controls access to existing behavior
- Both wrap an object with the same interface

Key Takeaways

1. Same interface as the real object
2. Multiple proxy types for different purposes
3. Proxies can be stacked
4. Client doesn't know it's using a proxy

Chapter 21: Bridge Pattern

Category: Structural

Intent

Decouple an abstraction from its implementation so both can vary independently.

Real-World Analogy

A remote control and a TV. Change remotes or TVs independently — they're connected by a "bridge" (infrared signal).

The Problem

Shapes that can be rendered in different ways. Without the bridge, M shapes x N renderers = M*N classes.

```
"""
================================================================
 BAD EXAMPLE: Without the Bridge Pattern
================================================================

Shapes that can be rendered in different ways (vector, raster).

Problems:
    1. Class explosion: CircleVector, CircleRaster, SquareVector,
       SquareRaster... M shapes × N renderers = M*N classes!
    2. Adding a new shape -> N new classes (one per renderer)
    3. Adding a new renderer -> M new classes (one per shape)
    4. Two independent dimensions locked together via inheritance
================================================================
"""


class CircleVector:
    def draw(self):
        print("  Drawing circle as vector lines")

class CircleRaster:
    def draw(self):
        print("  Drawing circle as pixels")

class SquareVector:
    def draw(self):
        print("  Drawing square as vector lines")

class SquareRaster:
```

```
    def draw(self):
        print("  Drawing square as pixels")

# TriangleVector? TriangleRaster? 3D renderer?
# 3 shapes × 3 renderers = 9 classes!


if __name__ == "__main__":
    shapes = [CircleVector(), CircleRaster(), SquareVector(), SquareRaster()]
    for s in shapes:
        s.draw()

    print()
    print("2 shapes × 2 renderers = 4 classes.")
    print("5 shapes × 3 renderers = 15 classes!")
    print("-> The Bridge pattern fixes this.")
```

The Solution

Separate shape and renderer into two independent hierarchies connected by composition.

```
"""
============================================================
 DESIGN PATTERN #21: BRIDGE
============================================================

Category: Structural
Intent:   Decouple an abstraction from its implementation so that
          the two can vary independently.

Real-world analogy:
    A remote control and a TV. The remote (abstraction) works with
    any TV brand (implementation). You can change remotes or TVs
    independently — they're connected by a "bridge".

When to use:
    - You have two independent dimensions that vary
    - Class explosion from combining dimensions via inheritance
    - You want to switch implementations at runtime

============================================================
"""

from abc import ABC, abstractmethod
```

```python
# ---------------------------------------------------------
# STEP 1: Implementation interface (the "how")
# ---------------------------------------------------------

class Renderer(ABC):
    @abstractmethod
    def render_circle(self, x: int, y: int, radius: int) -> None:
        pass

    @abstractmethod
    def render_rectangle(self, x: int, y: int, w: int, h: int) -> None:
        pass


# ---------------------------------------------------------
# STEP 2: Concrete implementations
# ---------------------------------------------------------

class VectorRenderer(Renderer):
    def render_circle(self, x: int, y: int, radius: int) -> None:
        print(f"  [PEN]  Vector circle at ({x},{y}) r={radius}")

    def render_rectangle(self, x: int, y: int, w: int, h: int) -> None:
        print(f"  [PEN]  Vector rect at ({x},{y}) {w}x{h}")


class RasterRenderer(Renderer):
    def render_circle(self, x: int, y: int, radius: int) -> None:
        print(f"  [PAINT] Raster circle at ({x},{y}) r={radius} (pixels)")

    def render_rectangle(self, x: int, y: int, w: int, h: int) -> None:
        print(f"  [PAINT] Raster rect at ({x},{y}) {w}x{h} (pixels)")


class SVGRenderer(Renderer):
    """New renderer — no shape classes modified!"""
    def render_circle(self, x: int, y: int, radius: int) -> None:
        print(f"  [SVG] <circle cx='{x}' cy='{y}' r='{radius}'/>")

    def render_rectangle(self, x: int, y: int, w: int, h: int) -> None:
        print(f"  [SVG] <rect x='{x}' y='{y}' width='{w}' height='{h}'/>")


# ---------------------------------------------------------
# STEP 3: Abstraction (the "what") — holds a reference to impl
# ---------------------------------------------------------
```

```python
class Shape(ABC):
    def __init__(self, renderer: Renderer):
        self._renderer = renderer  # THE BRIDGE!

    @abstractmethod
    def draw(self) -> None:
        pass


# ------------------------------------------------------------
# STEP 4: Refined abstractions
# ------------------------------------------------------------

class Circle(Shape):
    def __init__(self, renderer: Renderer, x: int, y: int, radius: int):
        super().__init__(renderer)
        self.x = x
        self.y = y
        self.radius = radius

    def draw(self) -> None:
        self._renderer.render_circle(self.x, self.y, self.radius)


class Rectangle(Shape):
    def __init__(self, renderer: Renderer, x: int, y: int, w: int, h: int):
        super().__init__(renderer)
        self.x = x
        self.y = y
        self.w = w
        self.h = h

    def draw(self) -> None:
        self._renderer.render_rectangle(self.x, self.y, self.w, self.h)


# ------------------------------------------------------------
# STEP 5: See it in action
# ------------------------------------------------------------

if __name__ == "__main__":
    renderers = [
        ("Vector", VectorRenderer()),
        ("Raster", RasterRenderer()),
        ("SVG", SVGRenderer()),
    ]
```

```python
    for name, renderer in renderers:
        print(f"=== {name} Renderer ===")
        circle = Circle(renderer, 10, 20, 5)
        rect = Rectangle(renderer, 0, 0, 100, 50)
        circle.draw()
        rect.draw()
        print()

    print(f"Total classes: 2 shapes + 3 renderers = 5 (not 6!)")
    print(f"5 shapes + 3 renderers = 8 classes (not 15!)")

    print()
    print("=" * 60)
    print("KEY TAKEAWAYS:")
    print("=" * 60)
    print("1. Shape and Renderer vary INDEPENDENTLY.")
    print("2. M shapes + N renderers = M+N classes (not M×N).")
    print("3. The 'bridge' is the renderer reference in Shape.")
    print("4. Can swap renderers at runtime.")
    print("5. Two separate hierarchies connected by composition.")
```

Structure

```
Abstraction (Shape)          Implementation (Renderer)
+--------------+             +----------------+
|    Shape     |--bridge---->|   Renderer     |
|   draw()     |             | render_circle() |
+--------------+             +----------------+
   ^       ^                    ^    ^    ^
   |       |                    |    |    |
Circle  Rectangle         Vector Raster  SVG
```

The Math

- Without Bridge: M shapes x N renderers = MxN classes
- With Bridge: M shapes + N renderers = M+N classes

Bridge vs Adapter

- **Adapter:** makes existing incompatible interfaces work together
- **Bridge:** designed upfront to let two dimensions vary independently

Key Takeaways

1. Two dimensions vary independently
2. M+N classes instead of MxN
3. The “bridge” is composition (not inheritance)

4. Can swap implementations at runtime

Chapter 22: Flyweight Pattern

Category: Structural

Intent

Use sharing to support large numbers of objects efficiently by separating shared (intrinsic) from unique (extrinsic) state.

Real-World Analogy

A forest in a game. Millions of trees, but only 5-10 tree types. Each tree shares its type's mesh/texture but has its own position.

The Problem

A text editor rendering thousands of characters. Each character object stores its own font, size, and color — massive duplication.

```
"""
==============================================================
 BAD EXAMPLE: Without the Flyweight Pattern
==============================================================

A text editor rendering thousands of characters. Each character
object stores its own font, size, and color.

Problems:
    1. 10,000 characters = 10,000 font/size/color copies
    2. Most characters share the same formatting
    3. Massive memory waste for duplicate data
==============================================================
"""

import sys


class BadCharacter:
    def __init__(self, char: str, font: str, size: int, color: str):
        self.char = char
        self.font = font      # duplicated thousands of times!
        self.size = size       # duplicated thousands of times!
        self.color = color     # duplicated thousands of times!
```

```python
if __name__ == "__main__":
    text = "Hello World! " * 1000  # 13,000 characters
    characters = []
    for ch in text:
        characters.append(BadCharacter(ch, "Arial", 12, "black"))

    print(f"  Characters created: {len(characters)}")
    print(f"  Each stores: char + font + size + color")
    print(f"  Memory per object: ~{sys.getsizeof(characters[0])} bytes")
    print(f"  Total objects: {len(characters)}")
    print()
    print("Every character stores its own copy of 'Arial', 12, 'black'")
    print("even though they're all the same!")
    print("-> The Flyweight pattern fixes this.")
```

The Solution

Store shared state in flyweight objects managed by a factory. Each context object only holds unique state.

```python
"""
============================================================
 DESIGN PATTERN #22: FLYWEIGHT
============================================================

Category: Structural
Intent:   Use sharing to support large numbers of fine-grained
          objects efficiently by separating intrinsic (shared)
          state from extrinsic (unique) state.

Real-world analogy:
    A forest in a video game. There are millions of trees, but
    only 5-10 tree TYPES (oak, pine, birch). Each tree shares
    the mesh/texture of its type (intrinsic) but has its own
    position/size (extrinsic).

Key terms:
    - Intrinsic state: shared, immutable (font, texture, mesh)
    - Extrinsic state: unique per instance (position, size)

When to use:
    - You have a HUGE number of similar objects
    - Objects share most of their state
    - Memory is a concern

============================================================
"""
```

```python
# ---------------------------------------------------------
# STEP 1: The Flyweight — stores shared (intrinsic) state
# ---------------------------------------------------------

class CharacterStyle:
    """Shared formatting — only created once per unique combination."""
    def __init__(self, font: str, size: int, color: str):
        self.font = font
        self.size = size
        self.color = color

    def render(self, char: str, x: int, y: int) -> str:
        return f"'{char}' at ({x},{y}) [{self.font} {self.size}px {self.color}]"


# ---------------------------------------------------------
# STEP 2: Flyweight Factory — manages shared instances
# ---------------------------------------------------------

class StyleFactory:
    _styles: dict[tuple, CharacterStyle] = {}

    @classmethod
    def get_style(cls, font: str, size: int, color: str) -> CharacterStyle:
        key = (font, size, color)
        if key not in cls._styles:
            cls._styles[key] = CharacterStyle(font, size, color)
            print(f"  [NEW] Created new style: {font} {size}px {color}")
        return cls._styles[key]

    @classmethod
    def style_count(cls) -> int:
        return len(cls._styles)


# ---------------------------------------------------------
# STEP 3: The context — stores unique (extrinsic) state
# ---------------------------------------------------------

class Character:
    """Each character only stores its unique data + a reference to shared style."""
    def __init__(self, char: str, x: int, y: int, style: CharacterStyle):
        self.char = char
        self.x = x
        self.y = y
        self.style = style  # shared reference, not a copy!
```

```python
    def render(self) -> str:
        return self.style.render(self.char, self.x, self.y)


# ------------------------------------------------------------
# STEP 4: See it in action
# ------------------------------------------------------------

if __name__ == "__main__":
    StyleFactory._styles.clear()

    print("=== Creating characters ===")
    # Get shared styles (only created once each)
    normal = StyleFactory.get_style("Arial", 12, "black")
    bold = StyleFactory.get_style("Arial-Bold", 12, "black")
    heading = StyleFactory.get_style("Arial-Bold", 24, "blue")

    # Create many characters sharing the same styles
    characters = []
    text = "Hello World"
    for i, ch in enumerate(text):
        style = heading if i == 0 else normal
        characters.append(Character(ch, i * 10, 0, style))

    # Add more text with the same styles
    text2 = "Design Patterns"
    for i, ch in enumerate(text2):
        style = bold if ch.isupper() else normal
        characters.append(Character(ch, i * 10, 20, style))

    print(f"\n=== Rendering ===")
    for ch in characters[:5]:
        print(f"  {ch.render()}")
    print(f"  ... and {len(characters) - 5} more characters")

    print(f"\n=== Memory savings ===")
    print(f"  Total characters: {len(characters)}")
    print(f"  Unique styles created: {StyleFactory.style_count()}")
    print(f"  Without flyweight: {len(characters)} style objects")
    print(f"  With flyweight: {StyleFactory.style_count()} style objects")
    print(f"  Saving: {len(characters) - StyleFactory.style_count()} duplicate objects!

    # Simulate a large document
    print(f"\n=== Large document simulation ===")
    StyleFactory._styles.clear()
    large_chars = []
    large_text = "Lorem ipsum dolor sit amet " * 1000
```

```python
    for i, ch in enumerate(large_text):
        style = StyleFactory.get_style("Arial", 12, "black")
        large_chars.append(Character(ch, i % 80 * 10, i // 80 * 15, style))

    print(f"  Characters: {len(large_chars)}")
    print(f"  Unique styles: {StyleFactory.style_count()}")
    print(f"  Style objects saved: {len(large_chars) - StyleFactory.style_count()}")

    print()
    print("=" * 60)
    print("KEY TAKEAWAYS:")
    print("=" * 60)
    print("1. Intrinsic (shared) state is stored ONCE in flyweights.")
    print("2. Extrinsic (unique) state is stored in each context.")
    print("3. Factory ensures flyweights are reused, not duplicated.")
    print("4. Huge memory savings with many similar objects.")
    print("5. Trade-off: slightly more complex code for big memory wins.")
```

Structure

```
+---------------+  get_style()  +-----------------+
| StyleFactory  | -------------> | CharacterStyle  |  (flyweight)
|               |               |  font, size,    |  <- shared!
| _styles: {}   |               |  color          |
+---------------+               +-----------------+
                                       ^
                                       | reference
+---------------+                      |
|  Character    |----------------------+
|  char, x, y  |  <- unique per instance
+---------------+
```

Key Terms

- **Intrinsic state:** shared, immutable (font, color, texture)
- **Extrinsic state:** unique per instance (position, size)

When to Use

- Huge number of similar objects
- Objects share most of their state
- Memory is a concern

Key Takeaways

1. Shared state stored once in flyweights
2. Unique state stored in each context object

3. Factory ensures reuse, not duplication
4. Huge memory savings
5. Trade-off: complexity for memory efficiency

Part IV: Architectural Patterns (Bonus)

Chapter 23: Repository Pattern

Category: Architectural / Data Access

Intent

Mediate between business logic and data access using a collection-like interface.

Real-World Analogy

A library. You ask the librarian to find/add/remove books. You don't go into the warehouse yourself.

The Problem

Business logic mixed directly with data access code. SQL scattered throughout business logic.

```
"""
==============================================================
 BAD EXAMPLE: Without the Repository Pattern
==============================================================

Business logic mixed directly with data access code.

Problems:
    1. SQL/database logic scattered throughout business code
    2. Can't switch databases without rewriting business logic
    3. Can't test business logic without a real database
    4. Violates Single Responsibility Principle
==============================================================
"""
```

```python
class BadUserService:
    def __init__(self):
        # Pretend this is a real database
        self._db = {
            1: {"id": 1, "name": "Alice", "email": "alice@mail.com"},
            2: {"id": 2, "name": "Bob", "email": "bob@mail.com"},
        }

    def get_user(self, user_id: int):
        # Business logic mixed with data access!
        print(f"  SELECT * FROM users WHERE id = {user_id}")
        return self._db.get(user_id)

    def create_user(self, name: str, email: str):
        # More SQL mixed in...
        new_id = max(self._db.keys()) + 1
        print(f"  INSERT INTO users (name, email) VALUES ('{name}', '{email}')")
        self._db[new_id] = {"id": new_id, "name": name, "email": email}
        return self._db[new_id]

    def get_active_users(self):
        # What if we switch from SQL to MongoDB? Rewrite everything!
        print("  SELECT * FROM users WHERE active = true")
        return list(self._db.values())


if __name__ == "__main__":
    service = BadUserService()
    print(f"  User: {service.get_user(1)}")
    print(f"  New: {service.create_user('Charlie', 'charlie@mail.com')}")

    print()
    print("SQL is embedded in business logic.")
    print("Switch to MongoDB? Rewrite the entire service.")
    print("Write tests? Need a real database.")
    print("-> The Repository pattern fixes this.")
```

The Solution

Define a repository interface. Business logic depends on the interface, not any specific storage implementation.

```python
"""
================================================================
 DESIGN PATTERN #23: REPOSITORY (Bonus)
================================================================
```

```
Category: Architectural / Data Access
Intent:   Mediate between the domain/business logic and data
          mapping layers using a collection-like interface.

Real-world analogy:
    A library. You don't go into the warehouse to find a book.
    You ask the librarian (repository) to find, add, or remove
    books. The librarian knows WHERE books are stored; you don't.

When to use:
    - You want to separate business logic from data access
    - You need to swap data sources (SQL -> NoSQL -> API -> file)
    - You want testable code (mock the repository in tests)

================================================================
"""

from abc import ABC, abstractmethod
from dataclasses import dataclass


# ------------------------------------------------------------
# STEP 1: Domain model (pure business object)
# ------------------------------------------------------------

@dataclass
class User:
    id: int
    name: str
    email: str


# ------------------------------------------------------------
# STEP 2: Repository interface
# ------------------------------------------------------------

class UserRepository(ABC):
    @abstractmethod
    def get_by_id(self, user_id: int) -> User | None:
        pass

    @abstractmethod
    def get_all(self) -> list[User]:
        pass

    @abstractmethod
    def save(self, user: User) -> None:
```

```python
        pass

    @abstractmethod
    def delete(self, user_id: int) -> None:
        pass


# ------------------------------------------------------------
# STEP 3: Concrete implementations
# ------------------------------------------------------------

class InMemoryUserRepository(UserRepository):
    """For development and testing."""
    def __init__(self):
        self._store: dict[int, User] = {}
        self._next_id = 1

    def get_by_id(self, user_id: int) -> User | None:
        return self._store.get(user_id)

    def get_all(self) -> list[User]:
        return list(self._store.values())

    def save(self, user: User) -> None:
        if user.id == 0:
            user.id = self._next_id
            self._next_id += 1
        self._store[user.id] = user
        print(f"  [SAVE] [InMemory] Saved: {user}")

    def delete(self, user_id: int) -> None:
        if user_id in self._store:
            removed = self._store.pop(user_id)
            print(f"  [DEL]  [InMemory] Deleted: {removed}")


class FileUserRepository(UserRepository):
    """Simulates file-based storage."""
    def __init__(self):
        self._data: dict[int, User] = {}

    def get_by_id(self, user_id: int) -> User | None:
        print(f"  [DIR] [File] Reading user {user_id} from disk...")
        return self._data.get(user_id)

    def get_all(self) -> list[User]:
        print(f"  [DIR] [File] Reading all users from disk...")
```

```
        return list(self._data.values())

    def save(self, user: User) -> None:
        self._data[user.id] = user
        print(f"  [DIR] [File] Wrote user to disk: {user}")

    def delete(self, user_id: int) -> None:
        if user_id in self._data:
            self._data.pop(user_id)
            print(f"  [DIR] [File] Deleted user {user_id} from disk")


# ------------------------------------------------------------
# STEP 4: Business logic — depends on interface, not impl
# ------------------------------------------------------------

class UserService:
    def __init__(self, repo: UserRepository):
        self._repo = repo  # depends on abstraction!

    def register_user(self, name: str, email: str) -> User:
        user = User(id=0, name=name, email=email)
        self._repo.save(user)
        return user

    def find_user(self, user_id: int) -> User | None:
        return self._repo.get_by_id(user_id)

    def remove_user(self, user_id: int) -> None:
        self._repo.delete(user_id)

    def list_users(self) -> list[User]:
        return self._repo.get_all()


# ------------------------------------------------------------
# STEP 5: See it in action
# ------------------------------------------------------------

if __name__ == "__main__":
    # Using in-memory repository (for dev/test)
    print("=== InMemory Repository ===")
    repo = InMemoryUserRepository()
    service = UserService(repo)

    service.register_user("Alice", "alice@mail.com")
    service.register_user("Bob", "bob@mail.com")
```

```python
    service.register_user("Charlie", "charlie@mail.com")

    print(f"\n  All users: {service.list_users()}")
    print(f"  Find #2: {service.find_user(2)}")

    service.remove_user(2)
    print(f"  After delete: {service.list_users()}")

    # Swap to file repository — business logic unchanged!
    print("\n=== File Repository (same business logic!) ===")
    file_repo = FileUserRepository()
    file_service = UserService(file_repo)

    file_service.register_user("Diana", "diana@mail.com")
    file_service.find_user(0)

    print()
    print("=" * 60)
    print("KEY TAKEAWAYS:")
    print("=" * 60)
    print("1. Business logic (UserService) has ZERO data access code.")
    print("2. Swap InMemory -> File -> SQL -> API by changing one line.")
    print("3. Testing: use InMemoryRepository (no real DB needed).")
    print("4. Each repository handles its own storage details.")
    print("5. This is the foundation of clean architecture.")
```

Structure

```
+-------------+  uses     +----------------+
| UserService | ---------> | UserRepository | (interface)
| (business)  |           |  get_by_id()   |
|             |           |  save()        |
| zero SQL!   |           |  delete()      |
+-------------+           +----------------+
                              ^         ^
                              |         |
                    +----------+  +----------+
                    | InMemory |  |   File   |
                    |   Repo   |  |   Repo   |
                    +----------+  +----------+
```

When to Use

- Separate business logic from data access
- Need to swap data sources
- Want testable code (mock the repo)

Key Takeaways

1. Business logic has zero data access code
2. Swap storage by changing one line
3. Test with in-memory repo (no real DB)
4. Foundation of clean architecture

Chapter 24: Dependency Injection

Category: Architectural / Creational

Intent

Instead of a class creating its own dependencies, they are provided ("injected") from outside.

Real-World Analogy

A chef doesn't grow vegetables. Ingredients are delivered. The chef works with any supplier — they just need ingredients that meet the spec.

The Problem

A notification service that creates its own dependencies. EmailSender is hardcoded — can't swap, can't test, can't configure.

```
"""
================================================================
 BAD EXAMPLE: Without Dependency Injection
================================================================

A notification service that creates its own dependencies.

Problems:
    1. EmailSender is hardcoded — can't swap to SMS or Slack
    2. Can't test NotificationService without sending real emails
    3. Tight coupling — changing EmailSender breaks everything
    4. No flexibility — one fixed configuration
================================================================
"""


class EmailSender:
    def send(self, to: str, message: str):
        print(f"  [EMAIL] Sending email to {to}: '{message}'")
```

```python
class BadNotificationService:
    def __init__(self):
        # Creates its OWN dependency — hardcoded!
        self.sender = EmailSender()

    def notify(self, user: str, message: str):
        self.sender.send(user, message)
        # Want to send via SMS instead? Must modify THIS class!
        # Want to test without sending real emails? Can't!


if __name__ == "__main__":
    service = BadNotificationService()
    service.notify("alice@mail.com", "Your order shipped!")
    service.notify("bob@mail.com", "Password reset requested")

    print()
    print("EmailSender is hardcoded inside NotificationService.")
    print("Can't swap, can't test, can't configure.")
    print("-> Dependency Injection fixes this.")
```

The Solution

Inject dependencies from outside. The service works with any implementation that meets the interface.

```python
"""
================================================================
 DESIGN PATTERN #24: DEPENDENCY INJECTION (Bonus)
================================================================

Category: Architectural / Creational
Intent:   Instead of a class creating its own dependencies,
          they are "injected" from outside. This inverts
          the control of dependency creation.

Real-world analogy:
    A restaurant chef doesn't grow their own vegetables.
    Ingredients (dependencies) are DELIVERED to the kitchen.
    The chef can work with any supplier — they just need
    ingredients that meet the spec.

Three types of injection:
    1. Constructor injection (most common, recommended)
    2. Method injection (per-call flexibility)
    3. Property injection (optional dependencies)
```

```python
When to use:
    - You want loosely coupled, testable code
    - You want configurable behavior without code changes

================================================================
"""

from abc import ABC, abstractmethod


# ----------------------------------------------------------
# STEP 1: Define abstractions (interfaces)
# ----------------------------------------------------------

class MessageSender(ABC):
    @abstractmethod
    def send(self, to: str, message: str) -> None:
        pass


class Logger(ABC):
    @abstractmethod
    def log(self, message: str) -> None:
        pass


# ----------------------------------------------------------
# STEP 2: Concrete implementations
# ----------------------------------------------------------

class EmailSender(MessageSender):
    def send(self, to: str, message: str) -> None:
        print(f"  [EMAIL] Email to {to}: '{message}'")


class SMSSender(MessageSender):
    def send(self, to: str, message: str) -> None:
        print(f"  [SMS] SMS to {to}: '{message}'")


class SlackSender(MessageSender):
    def send(self, to: str, message: str) -> None:
        print(f"  [CHAT] Slack to {to}: '{message}'")


class ConsoleLogger(Logger):
```

```python
    def log(self, message: str) -> None:
        print(f"  [LOG] LOG: {message}")


class NullLogger(Logger):
    """Silent logger — useful for tests."""
    def log(self, message: str) -> None:
        pass  # intentionally silent


# ------------------------------------------------------------
# STEP 3: Service with injected dependencies
# ------------------------------------------------------------

class NotificationService:
    # Constructor injection — dependencies provided from outside
    def __init__(self, sender: MessageSender, logger: Logger):
        self._sender = sender
        self._logger = logger

    def notify(self, user: str, message: str) -> None:
        self._sender.send(user, message)
        self._logger.log(f"Notified {user}: '{message}'")

    def broadcast(self, users: list[str], message: str) -> None:
        for user in users:
            self.notify(user, message)
        self._logger.log(f"Broadcast to {len(users)} users complete")


# ------------------------------------------------------------
# STEP 4: A simple DI container (optional, for larger apps)
# ------------------------------------------------------------

class Container:
    """A minimal dependency injection container."""
    def __init__(self):
        self._factories = {}

    def register(self, interface: type, factory) -> None:
        self._factories[interface] = factory

    def resolve(self, interface: type):
        if interface not in self._factories:
            raise KeyError(f"No registration for {interface}")
        return self._factories[interface]()
```

```python
# ------------------------------------------------------------
# STEP 5: See it in action
# ------------------------------------------------------------

if __name__ == "__main__":
    users = ["alice@mail.com", "bob@mail.com", "charlie@mail.com"]

    # --- Configuration 1: Email + Logging ---
    print("=== Email notifications with logging ===")
    service = NotificationService(
        sender=EmailSender(),
        logger=ConsoleLogger(),
    )
    service.notify("alice@mail.com", "Your order shipped!")

    # --- Configuration 2: SMS + No logging ---
    print("\n=== SMS notifications (silent logging) ===")
    service = NotificationService(
        sender=SMSSender(),
        logger=NullLogger(),  # no logging in production
    )
    service.notify("010-1234-5678", "Verification code: 1234")

    # --- Configuration 3: Slack + Logging ---
    print("\n=== Slack broadcast ===")
    service = NotificationService(
        sender=SlackSender(),
        logger=ConsoleLogger(),
    )
    service.broadcast(["#general", "#dev", "#ops"], "Deploy v2.0 complete!")

    # --- Using a DI Container ---
    print("\n=== Using DI Container ===")
    container = Container()
    container.register(MessageSender, lambda: EmailSender())
    container.register(Logger, lambda: ConsoleLogger())

    service = NotificationService(
        sender=container.resolve(MessageSender),
        logger=container.resolve(Logger),
    )
    service.notify("diana@mail.com", "Welcome!")

    print()
    print("=" * 60)
    print("KEY TAKEAWAYS:")
    print("=" * 60)
```

```
    print("1. Dependencies are INJECTED, not created internally.")
    print("2. Same NotificationService works with Email, SMS, or Slack.")
    print("3. Testing: inject NullLogger or mock sender — no real I/O.")
    print("4. Configuration happens at the 'composition root' (main).")
    print("5. DI Container automates dependency wiring for large apps.")
    print("6. This is arguably THE most important pattern for")
    print("   writing maintainable, testable code.")
```

Three Types of Injection

```
# 1. Constructor injection (recommended)
class Service:
    def __init__(self, sender: MessageSender):
        self._sender = sender

# 2. Method injection (per-call)
class Service:
    def send(self, sender: MessageSender, msg: str):
        sender.send(msg)

# 3. Property injection (optional deps)
service = Service()
service.logger = ConsoleLogger()
```

Structure

```
+------------+  injects  +---------------------+
|   main()   | ---------> |NotificationService  |
| (composer) |           |  sender: injected   |
|            |           |  logger: injected   |
+------------+           +---------------------+
                              uses v
                         +--------------+
                         |MessageSender | (interface)
                         +--------------+
                           ^    ^    ^
                         Email SMS  Slack
```

When to Use

- Want loosely coupled, testable code
- Need to swap implementations easily
- Want configurable behavior

Key Takeaways

1. Dependencies injected, not created internally
2. Same service works with any implementation
3. Testing: inject mocks or nulls
4. Configuration at the "composition root"
5. Arguably THE most important pattern for maintainability

Chapter 25: Event Sourcing

Category: Architectural

Intent

Store a sequence of events instead of just current state. Current state is derived by replaying events.

Real-World Analogy

An accountant's ledger. Every transaction is recorded. The balance is calculated from the ledger. Any dispute can be traced.

The Problem

A bank account that only stores the current balance. No history, no audit trail, no way to undo or replay.

```
"""
==============================================================
 BAD EXAMPLE: Without Event Sourcing
==============================================================

A bank account that only stores the current balance.

Problems:
    1. No history — can't see HOW you got to the current balance
    2. Can't audit: "who deposited $500 on Tuesday?"
    3. Can't undo or replay: state is overwritten each time
    4. If a bug corrupts the balance, data is lost forever
    5. Can't rebuild state from scratch
==============================================================
"""


class BadBankAccount:
    def __init__(self, owner: str):
```

```
        self.owner = owner
        self.balance = 0  # only stores current state!

    def deposit(self, amount: float):
        self.balance += amount
        print(f"  Deposited ${amount:.2f}. Balance: ${self.balance:.2f}")

    def withdraw(self, amount: float):
        self.balance -= amount
        print(f"  Withdrew ${amount:.2f}. Balance: ${self.balance:.2f}")


if __name__ == "__main__":
    account = BadBankAccount("Alice")
    account.deposit(1000)
    account.deposit(500)
    account.withdraw(200)
    account.deposit(300)
    account.withdraw(150)

    print(f"\n  Final balance: ${account.balance:.2f}")
    print()
    print("How did we get to $1450?")
    print("When was each transaction? Who authorized it?")
    print("Can we undo the $200 withdrawal?")
    print("If a bug sets balance to $0, can we recover?")
    print("-> Event Sourcing fixes this.")
```

The Solution

Store immutable events in an append-only log. Derive current state by replaying events. Support time travel.

```
"""
==============================================================
 DESIGN PATTERN #25: EVENT SOURCING (Bonus)
==============================================================

Category: Architectural
Intent:   Instead of storing just the current state, store a
          sequence of EVENTS that led to the current state.
          The current state is derived by replaying events.

Real-world analogy:
    An accountant's ledger. Instead of just writing "Balance: $1450",
    they record every transaction: deposit $1000, deposit $500,
    withdraw $200... The balance is CALCULATED from the ledger.
```

```
    If there's a dispute, you can trace every penny.

When to use:
    - You need a complete audit trail
    - You want to reconstruct state at any point in time
    - You need undo/redo or "time travel" debugging
    - Your domain is naturally event-based (banking, orders, etc.)

============================================================
"""

from dataclasses import dataclass, field
from datetime import datetime
from enum import Enum


# ------------------------------------------------------------
# STEP 1: Define events
# ------------------------------------------------------------

class EventType(Enum):
    ACCOUNT_CREATED = "account_created"
    MONEY_DEPOSITED = "money_deposited"
    MONEY_WITHDRAWN = "money_withdrawn"
    ACCOUNT_FROZEN = "account_frozen"


@dataclass(frozen=True)
class Event:
    type: EventType
    data: dict
    timestamp: str = field(default_factory=lambda: datetime.now().strftime("%H:%M:%S"))

    def __str__(self):
        return f"[{self.timestamp}] {self.type.value}: {self.data}"


# ------------------------------------------------------------
# STEP 2: Event Store — the append-only log
# ------------------------------------------------------------

class EventStore:
    def __init__(self):
        self._events: list[Event] = []

    def append(self, event: Event) -> None:
        self._events.append(event)
```

```python
        print(f"  [LOG] Event stored: {event}")

    def get_events(self) -> list[Event]:
        return list(self._events)

    def get_events_up_to(self, index: int) -> list[Event]:
        return self._events[:index + 1]


# ---------------------------------------------------------
# STEP 3: Aggregate — rebuilds state from events
# ---------------------------------------------------------

class BankAccount:
    def __init__(self, event_store: EventStore):
        self._store = event_store
        self.owner = ""
        self.balance = 0.0
        self.is_frozen = False

    # --- Commands (generate events) ---

    def create(self, owner: str) -> None:
        self._apply(Event(EventType.ACCOUNT_CREATED, {"owner": owner}))

    def deposit(self, amount: float) -> None:
        if self.is_frozen:
            print("  [x] Account is frozen!")
            return
        self._apply(Event(EventType.MONEY_DEPOSITED, {"amount": amount}))

    def withdraw(self, amount: float) -> None:
        if self.is_frozen:
            print("  [x] Account is frozen!")
            return
        if amount > self.balance:
            print(f"  [x] Insufficient funds! Balance: ${self.balance:.2f}")
            return
        self._apply(Event(EventType.MONEY_WITHDRAWN, {"amount": amount}))

    def freeze(self) -> None:
        self._apply(Event(EventType.ACCOUNT_FROZEN, {}))

    # --- Event application ---

    def _apply(self, event: Event) -> None:
        """Store the event and update state."""
```

```python
        self._store.append(event)
        self._handle(event)

    def _handle(self, event: Event) -> None:
        """Update state based on a single event."""
        if event.type == EventType.ACCOUNT_CREATED:
            self.owner = event.data["owner"]
        elif event.type == EventType.MONEY_DEPOSITED:
            self.balance += event.data["amount"]
        elif event.type == EventType.MONEY_WITHDRAWN:
            self.balance -= event.data["amount"]
        elif event.type == EventType.ACCOUNT_FROZEN:
            self.is_frozen = True

    # --- Rebuild state from events ---

    @classmethod
    def from_events(cls, event_store: EventStore) -> "BankAccount":
        """Reconstruct the account by replaying ALL events."""
        account = cls(event_store)
        for event in event_store.get_events():
            account._handle(event)
        return account

    @classmethod
    def from_events_at(cls, event_store: EventStore, index: int) -> "BankAccount":
        """Reconstruct state at a specific point in time."""
        account = cls(EventStore())  # empty store for replay
        for event in event_store.get_events_up_to(index):
            account._handle(event)
        return account

    def status(self) -> str:
        frozen = " [FROZEN]" if self.is_frozen else ""
        return f"  [STAT] {self.owner}: ${self.balance:.2f}{frozen}"


# ------------------------------------------------------------
# STEP 4: See it in action
# ------------------------------------------------------------

if __name__ == "__main__":
    store = EventStore()
    account = BankAccount(store)

    print("=== Transactions ===")
    account.create("Alice")
```

```python
    account.deposit(1000)
    account.deposit(500)
    account.withdraw(200)
    account.deposit(300)
    account.withdraw(150)

    print(f"\n{account.status()}")

    # Full audit trail!
    print("\n=== Audit Trail ===")
    for i, event in enumerate(store.get_events()):
        print(f"  [{i}] {event}")

    # Time travel — what was the state after event #3?
    print("\n=== Time Travel: State after event #3 ===")
    past_account = BankAccount.from_events_at(store, 3)
    print(past_account.status())

    # Rebuild from scratch — same result!
    print("\n=== Rebuild from events ===")
    rebuilt = BankAccount.from_events(store)
    print(rebuilt.status())

    # Try frozen account
    print("\n=== Freeze account ===")
    account.freeze()
    account.deposit(999)  # rejected!
    print(account.status())

    print()
    print("=" * 60)
    print("KEY TAKEAWAYS:")
    print("=" * 60)
    print("1. Events are the source of truth, not current state.")
    print("2. State is DERIVED by replaying events.")
    print("3. Complete audit trail — every change is recorded.")
    print("4. Time travel — reconstruct state at any point.")
    print("5. Events are immutable — append-only, never modified.")
    print("6. If state is corrupted, just replay events to rebuild.")
    print()
    print("[GRAD] CONGRATULATIONS! You've completed all 25 design patterns!")
```

Structure

```
Commands            Events (append-only)         State
---------          ------------------          ------
deposit(1000) ->   [MoneyDeposited: 1000]  ->   balance: 1000
```

```
deposit(500)  ->   [MoneyDeposited: 500]   ->   balance: 1500
withdraw(200) ->   [MoneyWithdrawn: 200]   ->   balance: 1300

State = replay(all events)
State at time T = replay(events up to T)
```

Event Sourcing vs Traditional CRUD

	CRUD	Event Sourcing
Stores	Current state only	All events
History	Lost on update	Complete audit trail
Undo	Difficult	Replay without that event
Debugging	"How did we get here?"	Replay and see
Storage	Less data	More data (append-only)

When to Use

- Need complete audit trail (banking, medical, legal)
- Want time-travel debugging
- Domain is naturally event-based
- Need to reconstruct state at any point

Key Takeaways

1. Events are the source of truth
2. State is derived by replaying events
3. Complete audit trail
4. Time travel — state at any point in history
5. Events are immutable, append-only

Appendix: Pattern Quick Reference

#	Pattern	Category	Intent	When to Use
1	Strategy	Behavioral	Swap algorithms at runtime	Multiple ways to do the same thing
2	Observer	Behavioral	Notify many objects of state changes	One-to-many event notification
3	Command	Behavioral	Encapsulate actions as objects	Undo/redo, queuing, logging actions
4	State	Behavioral	Change behavior based on internal state	Object with many state-dependent behaviors
5	Template Method	Behavioral	Define algorithm skeleton, vary steps	Same workflow, different details
6	Iterator	Behavioral	Traverse collections without exposing internals	Uniform access to different collections
7	Chain of Responsibility	Behavioral	Pass request along a handler chain	Multiple potential handlers, unknown in advance
8	Mediator	Behavioral	Centralize complex communication	Many objects with complex interactions
9	Memento	Behavioral	Capture and restore object state	Save/restore, checkpoints, snapshots

#	Pattern	Category	Intent	When to Use
10	Visitor	Behavioral	Add operations without modifying classes	Stable hierarchy, frequent new operations
11	Factory Method	Creational	Let subclasses decide what to create	Unknown concrete type at compile time
12	Abstract Factory	Creational	Create families of related objects	Consistent product families
13	Builder	Creational	Build complex objects step by step	Many optional parameters
14	Singleton	Creational	Ensure single instance	Shared resource (DB, config, logger)
15	Prototype	Creational	Clone existing objects	Expensive creation, many similar objects
16	Decorator	Structural	Add behavior dynamically via wrapping	Avoid class explosion from combinations
17	Adapter	Structural	Convert incompatible interfaces	Third-party library integration
18	Facade	Structural	Simplify complex subsystem interface	Complex multi-step subsystem operations
19	Composite	Structural	Tree structures with uniform interface	Part-whole hierarchies
20	Proxy	Structural	Control access to another object	Lazy loading, caching, access control
21	Bridge	Structural	Decouple abstraction from implementation	Two independent varying dimensions

#	Pattern	Category	Intent	When to Use
22	Flyweight	Structural	Share state among many similar objects	Huge number of similar objects
23	Repository	Architectural	Separate business logic from data access	Swappable data sources, testability
24	Dependency Injection	Architectural	Provide dependencies from outside	Loosely coupled, testable code
25	Event Sourcing	Architectural	Store events instead of current state	Audit trails, time travel, event-based domains

---

End of textbook.